“시간이 아깝지 않은”

SK네트웍스 Family AI캠프 교재10

강사 이영배

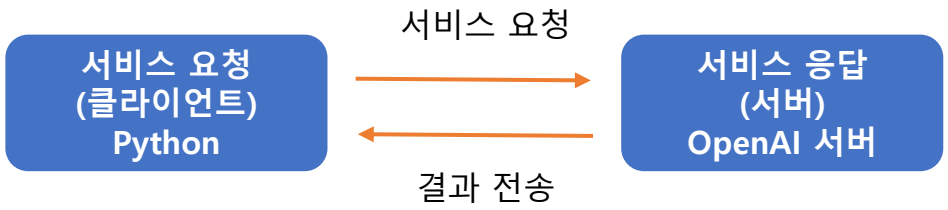
22. 생성형 AI의 언어 모델(LLM)

우리 목표

- 우리 회사 서비스에 AI 기능 넣기
  - ChatGPT, Gemini, Claude 웹 / 앱 서비스 직접 사용(X): 그들의 서비스
  - 우리에게 필요한 것: 웹 / 앱 서비스 화면(UI) 내부의 기능 (사용자 ↔ AI)

API(Application Programming Interface)

- API 개념
  - 서로 다른 컴퓨터 프로그램끼리 대화할 수 있게 돕는 메신저이자 통신 규칙
  - 반드시 OpenAI 등 서비스 업체에서 제공하는 규칙대로 구현하고 실행
  - 종류: REST API(웹 API), 라이브러리 API

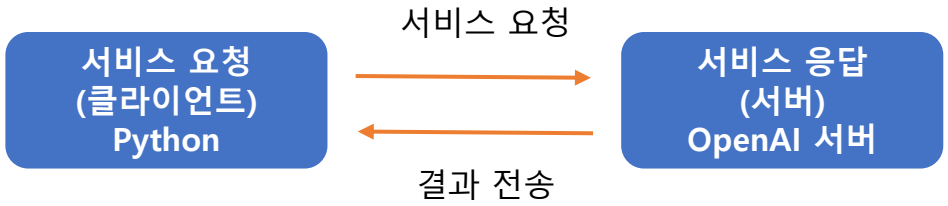



```
response = client.chat.completions.create(
    model="gpt-4o-mini",
    messages=[
        {'role':'system', 'content':'당신은 친절하고 정확한 AI 비서입니다.'},
        {"role": "user", "content": "최대 200 토큰 내에서 대규모 언어 모델에 대해서 알려줘"},
    ]
)
```

API(Application Programming Interface)

- JSON (JavaScript Object Notation) 정리

1) 개념: 서버와 클라이언트가 서로 데이터를 주고받을 때 사용하는 , 키-값 쌍으로 이루어진 텍스트 기반의 데이터 표준 형식




```
{
  "books": [
    {
      "book_id": 1,
      "title": "시간의 경계",
      "author": "이수진",
      "genre": "SF"
    },
    {
      "book_id": 2,
      "title": "사라진 기억",
      "author": "김민재",
      "genre": "미스터리"
    },
    {
      "book_id": 3,
      "title": "바람의 노래",
      "author": "정하늘",
      "genre": "판타지"
    }
  ]
}
```


2) python dict 자료형과 비교

	Python dict 자료형	JSON 문자열
구조	{key : value}	{key : value}
자료형	dict 자료형	string 자료형
따옴표 구분	작은따옴표('), 큰따옴표(") 모두 허용	반드시 큰따옴표(")만 사용
URL을 통한 데이터 전송 가능성	**불가능**	**가능**

OpenAI API 사용법

- API 필요성
  - **효율성**: 모든 기능을 직접 만들 필요 없이, 이미 잘 만들어진 타사의 기능(OpenAI의 LLM 대규모 언어 모델(LLM)을 사용하여 사용자의 메시지에 대한 응답을 생성)을 가져다 우리 회사 내 전용 Agentic AI 애플리케이션에 결합

OpenAI API 사용법

- OpenAI API 소개
  - OpenAI: 인공지능 연구 개발 기업, ChatGPT 개발
  - OpenAI API: 텍스트 생성(사용자 요청에 대한 응답), 코드 생성 등 다양한 기능 제공
  - API 키 발급 방법: OpenAI API 계정 생성 및 결재 후 API 키 발급

OpenAI API 사용법

- API 사용량 및 요금에 대한 이해
  - 핵심 단위: 토큰(Token)
  - 서브 워드 토큰화(subword tokenization)

    1) 개념: 단어를 고정된 단위로 자르는 것이 아니라, 의미를 가진 더 작은 단위인 '서브워드'로 분리하는 똑똑한 토큰화 방식

    2) 서브워드

    ① 단어(Word)를 더 쪼갠 의미를 가진 최소한의 글자 조각

    ② 필요성
    - 컴퓨터가 인간의 언어를 처리할 때, 신조어를 포함해서 세상의 모든 단어를 다 외우는 것은 불가능하다.
    - 서브워드는 컴퓨터가 적은 양의 단어 조각만 가지고도 조합하여 수억 개의 다양한 문장을 이해하고 처리할 수 있도록 돕는 레고 블록과 같다.



OpenAI API 사용법

3) 핵심 아이디어:

① 자주 사용되는 단어는 그 자체를 하나의 토큰으로 처리 (예: I, am, a)

② 자주 사용되지 않는 단어나 신조어는 의미를 가진 여러 개의 서브워드로 분리

- Chatbot → Chat + bot
- 먹었다 → 먹 + 었 + 다

4) 결론: 토큰(Token) 1개 = 서브워드 1개

구분	영어	한글	차이점
1 토큰당 글자 수	평균 **4글자** (알파벳 기준)	평균 **1~2글자** (공백 포함)	한글은 음절이나 자모 단위로 쪼개짐
단어당 토큰 수	1단어 ≈ **0.75 토큰**	1단어 ≈ **2~3 토큰**	한글은 조사와 어미 때문에 토큰이 증가함
비용 및 속도	기준점 (1배)	**약 2.5배 ~ 3배** 부담	한글 텍스트 처리가 훨씬 비싸고 느림

OpenAI API 사용법

- API 키 안전하게 관리하기
  - .env 파일: API 키와 같은 민감한 정보를 코드와 분리하여 별도의 파일에 저장하는 방법
  - .env 파일 생성 및 API 키
    ① 메모장에 .env 라는 이름으로 새 파일 생성
    ② 그 안에 다음과 같이 발급받은 API 키를 저장

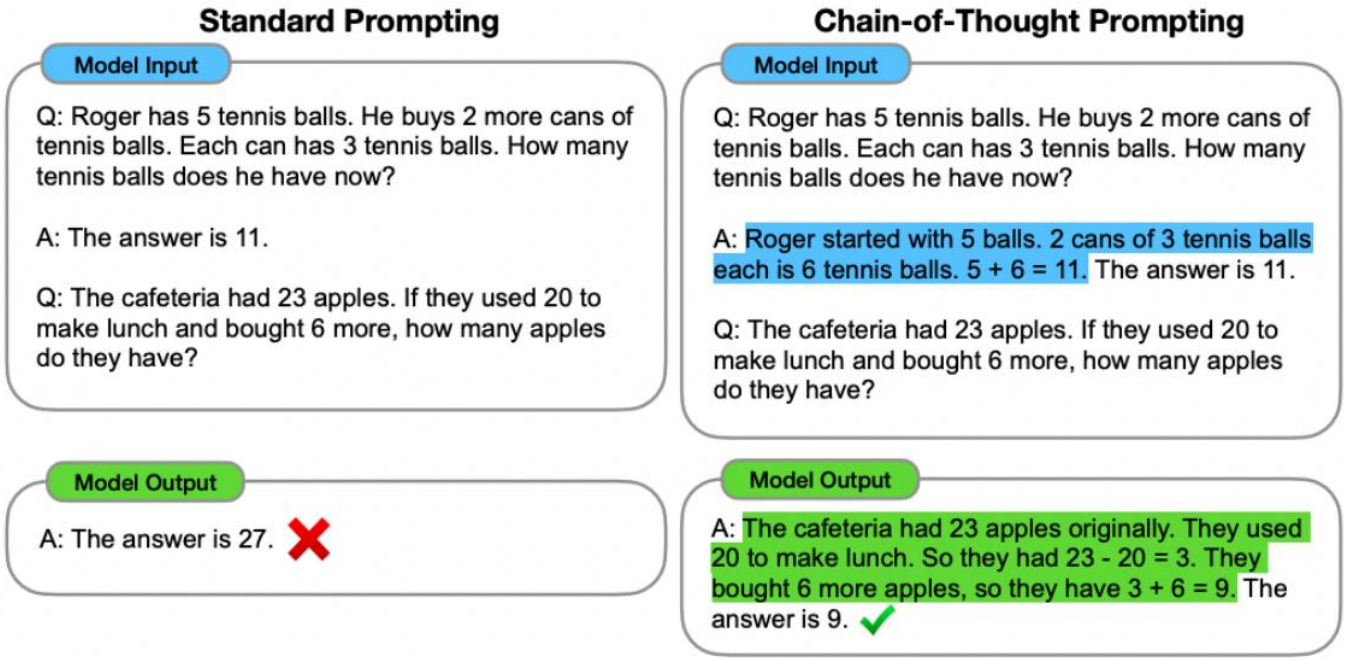
    OPENAI_API_KEY="sk-xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx"

실습: OpenAI API 사용법



OpenAI API 사용법: 프롬프트 엔지니어링(Prompt Engineering)

- 프롬프트 엔지니어링의 중요성
  - 프롬프트 (Prompt): 생성형 AI 모델에게 내리는 명령어 또는 질문
  - 프롬프트 엔지니어링 : 효과적인 프롬프트 작성 방법 연구
  - 중요성 : 생성형 AI 모델 성능 극대화, 원하는 결과 얻기 위해 필수



OpenAI API 사용법: 프롬프트 엔지니어링(Prompt Engineering)

- 효과적인 프롬프트 작성 방법
  - 구체적인 지시 : 명확하고 자세하게 요구사항 명시

    (예) "소설을 써줘" (X) → "등장인물 3명, 갈등 구조를 가진 로맨스 소설 1000자 내외로 써줘" (O)
  - 역할 부여 : AI 모델에게 특정 역할 부여

    (예) "너는 IT 전문가야. IT 트렌드를 요약해줘."
  - 맥락 제시 : 필요한 배경 정보 제공

    (예) "최근 경제 상황을 고려하여, 2026년 하반기 IT 시장 전망을 분석해줘."
  - 예시 활용 : 원하는 결과물의 예시를 제시

    (예) "다음과 같은 형식으로 뉴스 기사를 작성해줘: 제목, 본문, 출처"
  - 반복 및 개선: 결과물을 확인하며 프롬프트 수정 및 개선

실습: 프롬프트 엔지니어링(Prompt Engineering)

- 효과적인 프롬프트 작성 방법



실습: 고객 리뷰 감성 분석

- 감성 분석의 개념 및 활용 분야

1) 감성 분석 (Sentiment Analysis) : 텍스트에 나타난 감정(긍정, 부정, 중립) 분석

2) 활용 분야:

① 고객 리뷰 분석 : 쇼핑몰, 배달 앱 등에 남겨진 리뷰 분석, 제품/서비스 만족도 파악, 개선점 도출

② 소셜 미디어 분석 : 인스타그램, 블로그 등 SNS에서 브랜드나 특정 신제품에 대한 대중의 실시간 반응과 호감도 측정, 브랜드 평판 관리, 마케팅 전략 수립

실습: 고객 리뷰 감성 분석

- OpenAI API를 이용한 감성 분석 구현

  - 프롬프트 작성 전략

    ① "다음 고객 리뷰의 감성을 긍정, 부정, 중립 중 하나로 분류해줘: [고객 리뷰 텍스트]"

    ② 더 상세한 감성 분류 (매우 긍정, 긍정, 중립, 부정, 매우 부정) 가능

  - 프롬프트 작성 예시

```
# prompt 생성
user_prompt = f"""
다음 고객 리뷰의 내용을 분석하여 긍정 또는 부정으로 감성 분류하고, 해당 분류의
판단 근거가 되는 핵심 키워드들을 추출하여 JSON 형식으로 제시해 주세요

고객 리뷰:
{customer_review}

출력 형식:
아래 JSON 형식을 엄격하게 준수하여 결과를 생성해야 합니다.
{{
  "감성 분석 결과": "긍정" 또는 "부정",
  "판단 근거 키워드": {{
    "긍정": ["키워드1", "키워드2", ...],
    "부정": ["키워드1", "키워드2", ...]
  }}
}}
"""
```

실습: 고객 리뷰 감성 분석

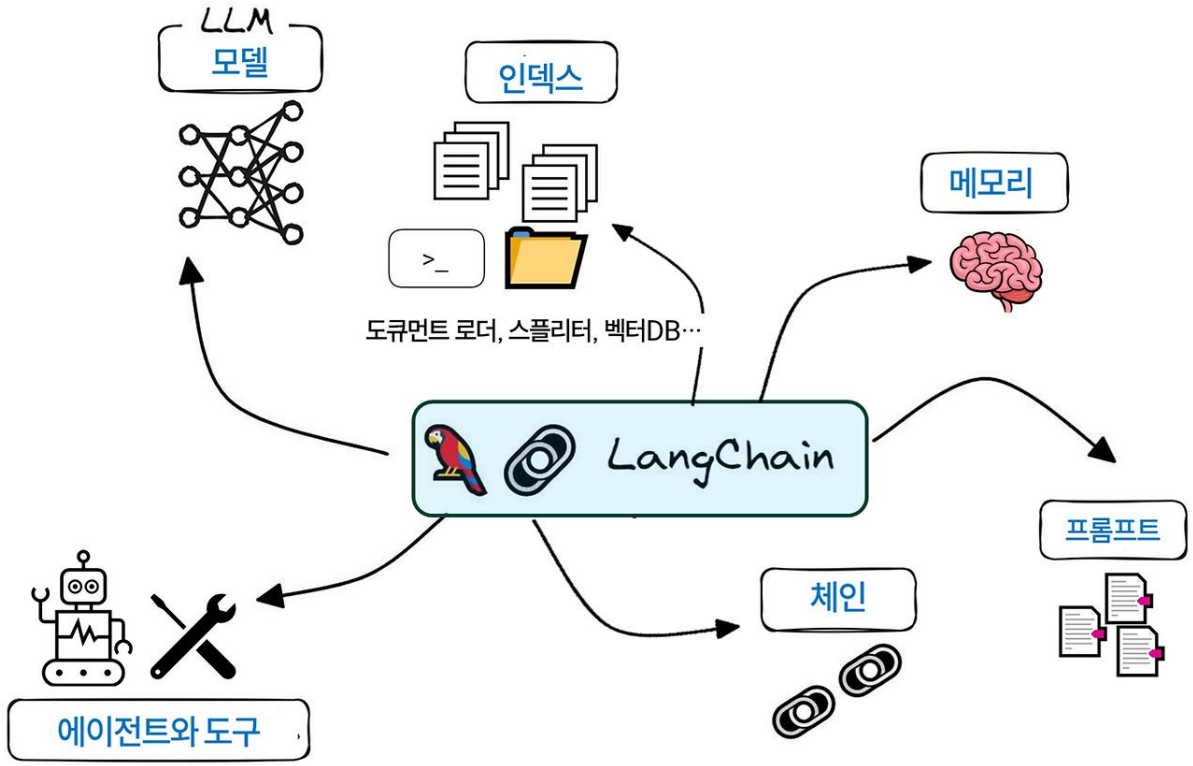
- OpenAI API를 이용한 감성 분석 구현



23. LangChain

LangChain

- 정의: 대규모 언어 모델(LLM)을 기반으로 한 AI 애플리케이션 개발을 빠르고 효율적으로 완성시켜주는 프레임워크
  - 목표: LLM의 추론 능력과 외부 데이터 소스 또는 기타 계산 도구를 결합하여 더욱 강력하고 유용한 애플리케이션을 생성



LangChain

- 라이브러리와 프레임워크의 차이점

라이브러리 (Library) : '요리 도구 세트 vs **프레임워크 (Framework) : '레고 집 만들기 세트'**

LangChain

- 기본 구조(Architecture): 체인 (Chain)

  1) 여러 구성 요소를 선형적으로 연결하여 일련의 작업을 자동화하는 워크플로우 & 기본 구조

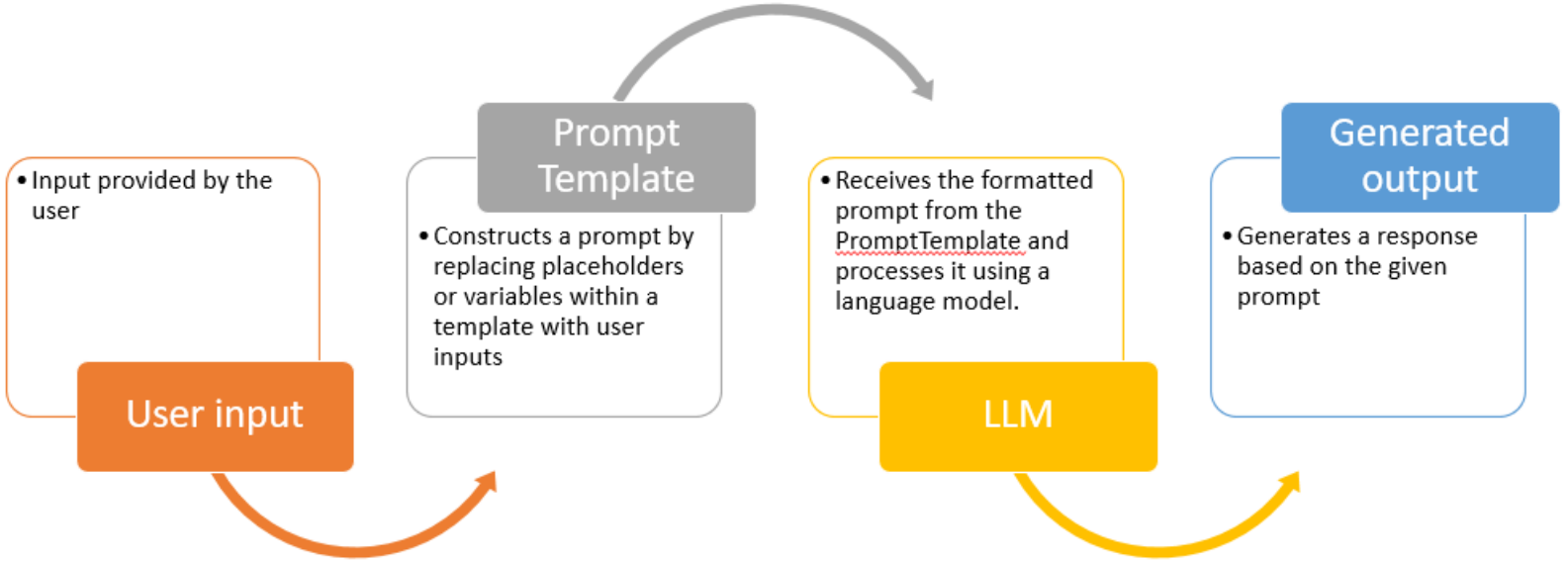
     예시) 사용자 질문에 답변하는 AI 챗봇 구현 → 프롬프트 + LLM + 출력 파서(AI의 출력 가공)

  2) LCEL(LangChain Expression Language)라는 문법을 사용하여 Chain 표현 및 생성

     예시) chain = 프롬프트 생성 함수 | LLM 생성 함수 | 출력 파서 생성 함수

실습 : Chain 생성 및 실행 실습

- 실습 개요
  1) LLM 모델 생성
  2) 프롬프트 생성
  3) Output Parser 생성
  4) Chain 생성
  5) Chain 실행



실습 : Chain 생성 및 실행 실습



실습 : 고객 리뷰 데이터를 이용한 감성 분석 애플리케이션 구현



실습 : 챗봇 구현

- 실습 개요
  1) 환경 설정
  2) LLM 모델 생성
  3) 대화형 프롬프트 템플릿 정의 (MessagesPlaceholder 활용)
  4) Chain 생성
  5) 챗봇 작동

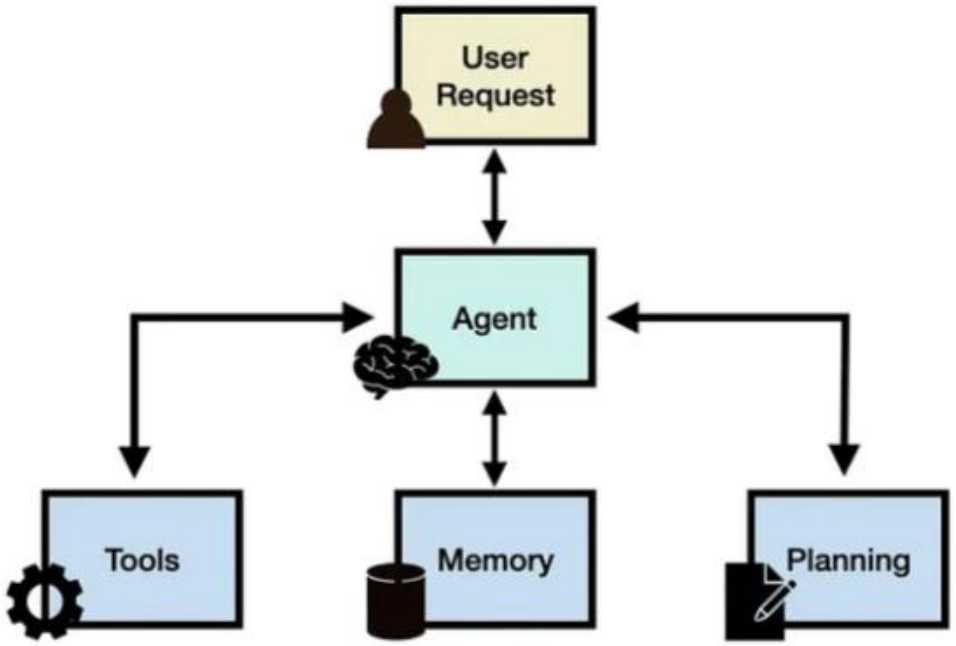
실습 : 챗봇 구현



AI 에이전트와 도구

- Agent

1) 개념: LLM을 두뇌로 해서 사용자의 의도를 분석한 후 목표 달성에 가장 적합한 도구를 동적으로 선택하고 호출함으로써 복잡한 작업을 수행하는 자율적인 의사 결정 시스템



AI 에이전트와 도구

- 도구(Tool)

1) 개념: LLM이 텍스트 생성이라는 한계를 넘어, 외부 시스템(API, 함수, 데이터베이스 등)와 상호작용하고 실제 작업을 수행할 수 있도록 연결해 주는 기능

2) 도구의 구성 요소

① 이름(Name): 에이전트가 도구를 호출할 때 사용하는 고유한 식별자

② 설명(description): 도구에 대한 설명, 에이전트(LLM)는 이 설명을 읽고 언제 이 도구를 사용해야 하는지, 어떤 문제를 해결하는 데 적합한지 판단

③ 입력 스키마 (Input Schema): 함수를 실행하기 위해 필요한 매개변수의 종류와 입력값의 자료형(타입) 정의

3) 실시간 웹 검색 및 콘텐츠 추출 API 도구: TavilySearch

```
# 필요한 도구 임포트
from langchain_tavily import TavilySearch

# 검색 도구 생성
search_tool = TavilySearch(max_results=3, topic="news")
```

AI 에이전트와 도구

4) TavilySearch 매개변수

① topic: 'general' (일반 기술/학술), 'news' (최신 뉴스), 'finance' (금융)

② search_depth: 'basic' (빠른 검색), 'advanced' (심층 문맥 추출, 권장)

③ max_results: 검색 시 웹에서 가져올 핵심 출처(웹페이지)의 최대 개수 지정

④ include_answer: Tavily 자체 AI가 1차 요약한 답변 포함 여부

⑤ time_range: 'day', 'week', 'month', 'year' (특정 기간 내 문서 필터링)

5) API Key 발급

① Tavily 웹사이트 회원가입

② API key 발급

③ .env 파일에 등록

실습: AI 에이전트와 도구

- 도구(Tool) 사용 실습: Tavily Search API



실습: 실시간 외부 정보 검색 및 답변 완성 AI 에이전트 구현

- 실습 개요

1) 목표: 최신 OpenAI LLM이 자연어를 이해하고, 상황에 맞춰 스스로 도구 호출(Function Calling)을 결정하는 메커니즘을 학습한다.

2) 과정

① 환경 설정

② 도구(Tools) 생성

③ LLM (언어 모델) 생성: 도구 사용 능력 부여

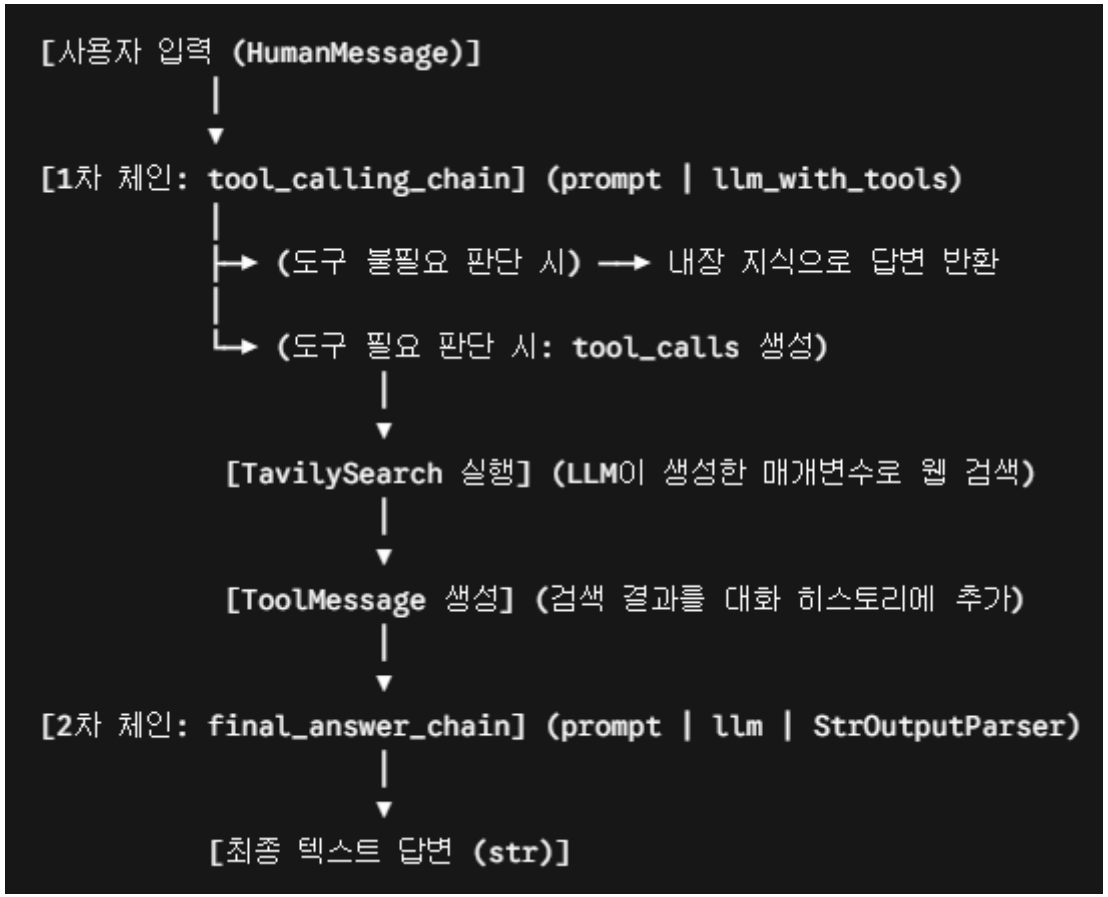
④ MessagesPlaceholder: LLM이 "내가 어떤 질문을 받았고, 어떤 도구 호출을 요청했으며, 그 결과로 어떤 데이터가 들어왔는가"를 한꺼번에 파악할 수 있는 메모리 기능 생성

⑤ tool_calling_chain, final_answer_chain → 2개의 chain 생성

⑥ AI 에이전트 실행 및 결과 분석

실습: 실시간 외부 정보 검색 및 답변 완성 AI 에이전트 구현

3) LCEL 기반 LLM - Tavily Search 도구 연결의 핵심 동작 흐름



실습: 실시간 외부 정보 검색 및 답변 완성 AI 에이전트 구현



24. RAG

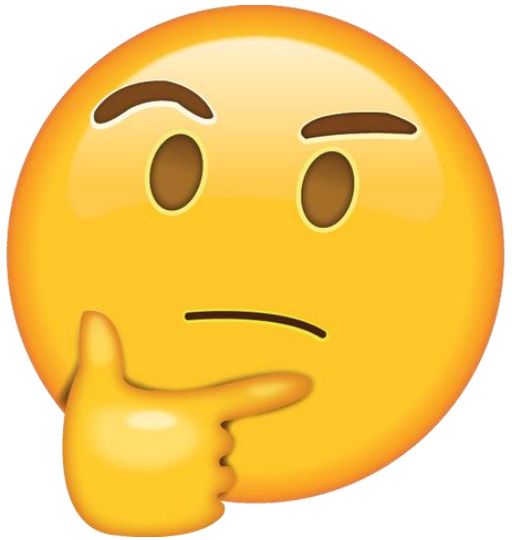
(Retrieval Augmented Generation)

생성형 AI : 빛과 그림자

보고서 작성, 아이디어 구상까지...



"이게 최선입니까? 확실해요?"



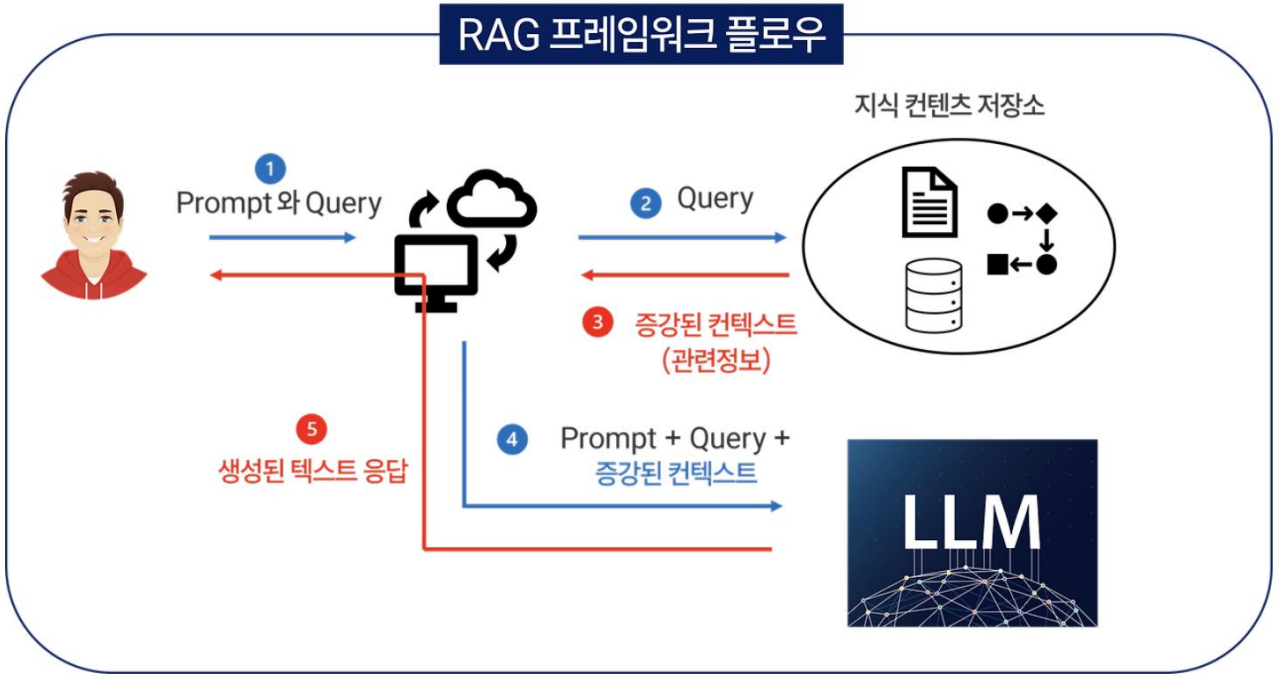
생성형 AI : 빛과 그림자

빛(놀라운 능력)	그림자(치명적 약점)
놀라운 창의력: 글, 그림, 코드 등 무엇이든 척척!	환각 (Hallucination): 그럴듯한 거짓말을 너무 잘해요!
방대한 지식: 세상의 온갖 정보를 학습했어요.	지식의 한계: 특정 시점까지의 데이터로 학습, 어제 나온 신제품은 몰라요.
유창한 소통: 사람처럼 자연스럽게 대화해요.	출처 불분명: 어떤 근거로 답하는지 알 수 없어 믿기 어려워요.

기업은 **부정확하고 신뢰할 수 없는 AI**를 고객 응대나
중요한 의사결정에 사용할 수 없음

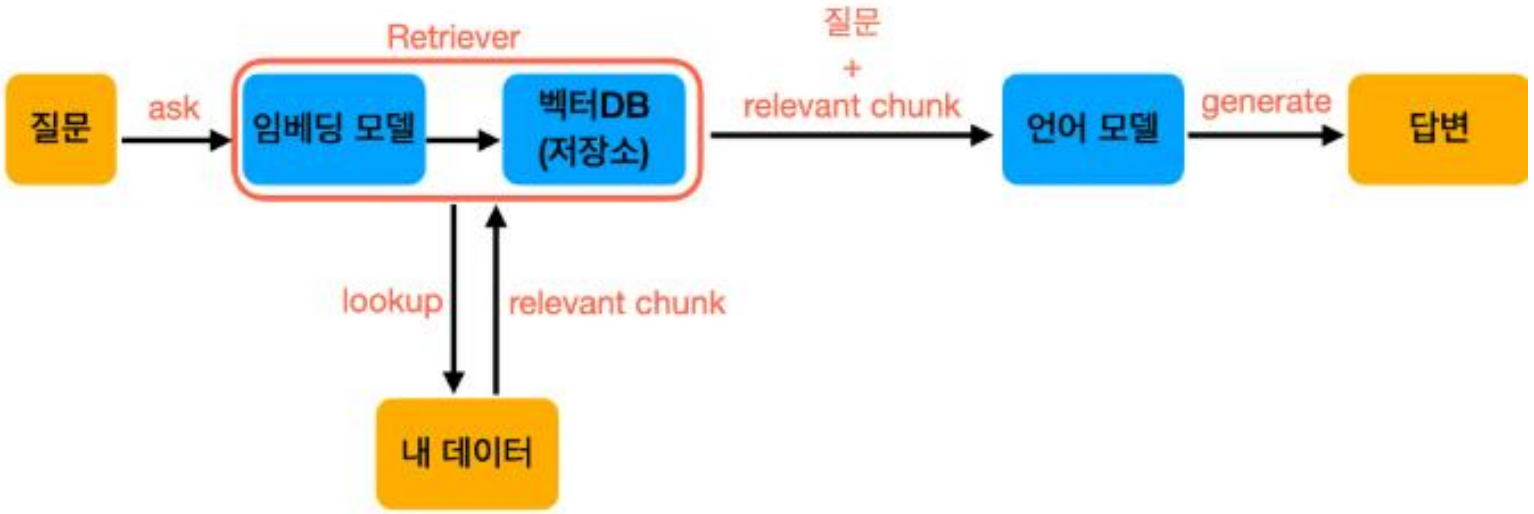
가장 현실적인 대안

- RAG (Retrieval Augmented Generation) system 도입
  1) RAG : 검색(Retrieval) + 증강(Augmented) + 생성(Generation) 의 약자
  2) RAG system: AI가 답변하기 전, 기업의 내부 데이터나 최신 웹 정보 같은 신뢰할 수 있는 자료를 먼저 '검색(Retrieval)'하고 이를 기반으로 답변을 생성(Generation)하여 AI의 약점을 보완하는 핵심 기술



실습: 지능형 고객 응대 Chatbot 구현

- 실습 개요
  - 목표: 고객 응대 매뉴얼을 기반으로 RAG 시스템을 구축하여 '지능형 고객 응대 전문 Chatbot'을 구현한다



실습: 지능형 고객 응대 Chatbot 구현

- 실습 개요

  - 과정

    1) 데이터 수집 및 변환: 텍스트 파일을 LangChain의 기본 자료형 Document 자료형으로 변환
    2) 텍스트 분할: 임베딩 모델이 한 번에 처리할 수 있는 입력 길이 제한을 고려하여 긴 문서를 작은 조각(chunk)로 분할
    3) 텍스트 임베딩
    4) 벡터 저장소 및 검색 도구 생성
       ① 텍스트와 임베딩 벡터들을 효율적으로 저장하고 검색할 수 있는 전용 DB인 벡터 저장소(VectorStore) 구축
       ② 벡터 저장소를 기반으로 사용자의 질문과 관련된 문서를 검색(Retrieval)하는 검색 도구 생성
    5) 프롬프트와 LLM 생성
    6) RAG(Retrieval-Augmented Generation) 체인(Chain) 구성
    7) Chatbot 실행

실습: 지능형 고객 응대 Chatbot 구현

- 벡터 데이터베이스(Vector DB)

1) 개념: 텍스트, 이미지 등의 비정형 데이터를 임베딩 모델을 통해 임베딩 벡터로 변환하여 저장하고, 유사도 연산을 수행하는 전용 데이터베이스

2) RAG(검색 증강 생성)에서의 핵심 역할

① 지식 컨텍스트 수집(Retrieval): LLM이 답변을 생성하기 전, 질문과 의미적으로 가장 가까운 문서를 저장된 데이터(문서와 임베딩 벡터)에서 초고속으로 검색

② 환각(Hallucination) 방지: LLM에게 최신 데이터나 기업 내부 데이터를 답변의 근거로 제공

3) 종류: FAISS, ChromaDB

실습: 지능형 고객 응대 Chatbot 구현

- 데이터 전처리 및 임베딩 실습



실습: 지능형 고객 응대 Chatbot 구현

- 벡터 저장소 구축 및 검색 도구 생성 실습



실습: 지능형 고객 응대 전문 Chatbot 구현

- RAG 완성: Chatbot 구현 및 실행 실습



밀집 검색(Dense Retrieval)과 키워드 기반 희소 검색(Sparse Search)

- 밀집 검색(Dense Retrieval)

  1) 개념: 텍스트를 SentenceTransformer와 같은 임베딩 모델을 통해 고차원의 연속적인 수치 벡터로 변환(임베딩)한 후, 코사인 유사도를 기반으로 가장 유사한 문서를 찾아내는 방식

  2) 텍스트 임베딩 기반의 RAG의 방식

- 키워드 기반 희소 검색(Sparse Retrieval)

  • 개념: 문장 내에 포함된 특정 단어(키워드)들의 등장 빈도와 전체 문서에서의 희귀성을 기반으로 사용자 질의(Query)와 문서 간의 연관성을 계산하여 관련성이 가장 높은 문서를 찾아내는 방식

키워드 기반 희소 검색(Sparse Search)

- BM25 (Best Matching 25) 알고리즘

1) 개념: TF-IDF 방식을 개선하여 문서 길이와 단어 빈도의 과도한 영향을 완화한 산업 표준 키워드 검색 알고리즘

2) 핵심 메커니즘

① TF (Term Frequency) 포화도 조절: 단어가 반복 등장할수록 가중치가 증가하지만, 일정 수준 이상부터는 가중치 상한선을 두어 특정 단어의 과도한 독점을 방지

② 문서 길이 정규화: 문장이 길다는 이유만으로 단어 일치 확률이 높아져 점수가 불공정하게 상승하는 현상을 보정

③ IDF (Inverse Document Frequency): 전체 문서에서 희귀하게 등장하는 단어일수록 높은 중요도 점수 부여

3) 한글 처리 시 핵심 고려사항

① 한글은 조사와 어미가 발달한 교착어이므로 단순 띄어쓰기 토큰화 시 검색 성능 급감

② 한국어 형태소 분석기(mecab, OKT 등)를 활용한 형태소 단위 토큰화 전처리가 필수적임

키워드 기반 희소 검색(Sparse Search)

- BM25 (Best Matching 25) 알고리즘

4) 검색 과정

[사전 작업: 인덱싱 단계]

원본 문서 데이터 → 형태소 분석 (토큰화) → 단어별 문서 위치 정리 (역색인, Inverted Index 구축)



[실시간 작업: 검색 단계]

사용자 질문 입력 → 형태소 분석 (질의 토큰화) → 연관 문서 필터링 → 문서별 BM25 점수 계산 (TF/IDF/길이 반영) ➔ 정렬 및 반환

키워드 기반 희소 검색(Sparse Search)

- BM25 (Best Matching 25) 알고리즘

5) 역색인(Inverted Index)

① 일반적인 데이터 저장 방식은 "문서 ID →그 문서에 포함된 단어들" 구조

② 역색인: "단어(Keyword) →그 단어가 포함된 문서 ID들의 목록" 형태로 저장하는 방식

③ 역색인 구축 과정 3단계 (예시)

- [기본 데이터]
  - 문서 1 (Doc 1): "철수가 밥을 먹었다"
  - 문서 2 (Doc 2): "영희가 밥을 샀다"
- 1단계: 토큰화 및 전처리 (형태소 분석)
  - 텍스트에서 조사(가, 을)를 제거하고 의미 있는 명사와 동사(원형)만 추출합니다.
  - Doc 1: 철수, 밥, 먹다
  - Doc 2: 영희, 밥, 사다

키워드 기반 희소 검색(Sparse Search)

- BM25 (Best Matching 25) 알고리즘

5) 역색인(Inverted Index)

- 2단계: 단어 중심의 매칭 구조로 뒤집기(역색인)
  - 각 단어가 어떤 문서(Doc ID)에서, 몇 번(TF, 단어 빈도) 등장했는지 역으로 추적

단어 (Term)	포함된 문서 목록	세부 정보 [문서 ID, 등장 빈도(TF)]
밥	Doc 1, Doc 2	[Doc 1, TF: 1], [Doc 2, TF: 1]
먹다	Doc 1	[Doc 1, TF: 1]
영희	Doc 2	[Doc 2, TF: 1]
사다	Doc 2	[Doc 2, TF: 1]

- 3단계: 최종 데이터베이스(인덱스) 완성
  - 이 표를 단어 사전 순으로 정렬하여 메모리와 디스크에 저장 → 역색인 파일

키워드 기반 희소 검색(Sparse Search)

- BM25 (Best Matching 25) 알고리즘

6) BM25 (Best Matching 25) 알고리즘 정리

- 사용자의 질문에서 핵심 키워드들을 추출한 뒤, 그 키워드가 포함된 문서들을 찾아낸다.
- 단순히 단어가 많이 들어있는 문서를 추출하는 것이 아니라,
  ① 그 단어가 얼마나 독특한지(IDF),
  ② 문서 안에서 얼마나 핵심적으로 쓰였는지(TF 포화도),
  ③ 문서의 분량이 쓸데없이 길지는 않은지(길이 정규화)를
- 종합적으로 점수화하여 가장 정답에 가까운 문서를 순서대로 정렬해 주는 수학적 검색 알고리즘이다.

키워드 기반 희소 검색(Sparse Search)

- 장점
  1) 고유명사, 제품 일련번호, 에러 코드, 규제 조항 등에 대한 검색의 뛰어난 정확도
  2) 별도의 임베딩 모델 학습이나 GPU 자원이 필요 없어 빠른 인덱싱 및 검색 속도 제공
- 단점
  1) 의미적 유사성(Semantic Similarity) 파악 불가 (단어의 외형적 일치만 검증)
  2) 동의어/유의어 미인식 문제 (예: '환불'과 '반품', '컴퓨터'와 'PC'를 서로 다른 단어로 취급)

실습: 키워드 기반 희소 검색(Sparse Search)



하이브리드 검색(Hybrid Search)

- 개념: 키워드 기반의 '희소 검색(Sparse Search)'과 의미/문맥 기반의 '밀집 검색(Dense Search)'을 결합하여 양쪽의 장점을 모두 취하는 통합 검색 방식

- 희소 검색 vs 밀집 검색 비교

구분	희소 검색 (Sparse / BM25)	밀집 검색 (Dense / Vector DB)
검색 기준	키워드 외형적 완전 일치	문맥적 의미 유사성
강점	고유명사, 제품명, 숫자, 코드	동의어, 유의어, 문맥 이해
약점	문맥 미인식, 동의어 처리 불가	환각, 정확한 고유명사 누락

하이브리드 검색(Hybrid Search)

- 하나의 검색 방식만 사용할 경우 RAG 시스템의 문제점

1) 밀집 검색만 쓸 때의 문제: 사용자가 "오류 코드 X-2201 원인이 뭐야?"라고 질문하면, 임베딩 모델은 전체적인 문맥적 의미만 파악할 뿐, 정작 핵심인 'X-2201'이라는 고유 식별자를 무시하고 유사한 표현의 다른 문서("오류 코드 X-2202 원인이 뭐야?")를 가져올 수 있다.

2) 희소 검색만 쓸 때의 문제: 사용자가 "컴퓨터 화면이 안 켜져요"라고 물었을 때, 문서에 "모니터 전원 공급 불량"이라는 해결책이 있어도 단어가 일치하지 않아 검색에 실패한다.

- 하이브리드 RAG 도입 효과

• 정밀한 단어 일치와 문맥 검색을 동시에 충족하여 단일 검색 대비 RAG 성능 향상 가능

하이브리드 결과 통합 메커니즘: RRF (Reciprocal Rank Fusion)

- RRF (Reciprocal Rank Fusion / 상호 순위 융합)
  1) 개념: 서로 다른 스케일과 점수 체계를 가진 희소 검색과 밀집 검색의 결과를 각 검색기에서의 '순위(Rank)'만을 기반으로 공정하게 재정렬하여 결합하는 알고리즘
     ① 희소 검색(BM25): 점수 상한선이 없고 단어 빈도에 따라 15.4, 32.1 등 임의의 양수가 나온다.
     ② 밀집 검색(Dense): 보통 코사인 유사도를 쓰기 때문에 0.75, 0.88 등 0과 1 사이의 실수가 나온다.
     ③ 두 점수는 척도(Scale)가 달러서 단순히 더하면(예: 32.1 + 0.88) 점수대가 큰 BM25 결과에 밀집 검색 결과가 완전히 묻혀버린다.

  2) RRF는 이 점수들을 과감히 버리고 오직 문서의 '등수(순위, Rank)'만을 사용하여, 두 검색 엔진에서 골고루 상위에 오른 문서가 가장 높은 점수를 받도록 만든다.

- LangChain 구현 방식: EnsembleRetriever
  1) **EnsembleRetriever**(retrievers=[bm25_retriever, dense_retriever], weights=[0.5, 0.5])
  2) 두 검색기 결과에 가중치(Weights)를 부여하여 통합 순위를 계산하고 최적의 컨텍스트 도출

실습: 하이브리드 검색(Hybrid Search)



FAISS vs ChromaDB 핵심 차이 비교

- FAISS (Facebook AI Similarity Search)

1) 개념 및 특징

① Meta(Facebook) AI Research에서 개발한 C++ 기반의 초고속 벡터 유사도 검색 및 클러스터링 라이브러리

② 단순 DB 형태라기보다는 메모리(In-Memory) 기반의 고성능 검색 엔진(Engine)에 가까움

2) 장점

① 압도적인 연산 속도 및 메모리 효율성: 벡터 유사도 연산 속도가 매우 빠르다.

② 경량성: 별도의 DB 서버 구축 없이 파이썬 라이브러리 설치만으로 즉시 구동 가능

③ 연구 및 프로토타이핑 표준

FAISS vs ChromaDB 핵심 차이 비교

- FAISS (Facebook AI Similarity Search)

3) 단점

① 기본적으로 메모리 상에서 작동하며, 디스크 저장/복원 시 명시적인 파일 저장(save) 및 로드(load) 처리 필요

② SQL의 'where '와 같은 조건별 검색 기능에 해당하는 '메타데이터 검색 기'능이 상대적으로 제한적(어려움)

③ 메타데이터 검색 기능: 문서에 성별, 날짜, 부서, 카테고리 같은 '메타데이터(조건)'를 결합하여 원하는 데이터만 정확하게 도려내듯 찾아내는 기능

4) 핵심 사용법 (LangChain 기준)

① 벡터 저장소(vectorstore) 생성: vectorstore = FAISS.from_documents(docs, model)

② 검색 도구 생성: retriever = vectorstore.as_retriever(search_kwargs={'k': 2})

FAISS vs ChromaDB 핵심 차이 비교

- ChromaDB

1) 개념 및 특징

① AI 애플리케이션 및 LLM 개발자를 위해 설계된 오픈소스 임베딩 전용 데이터베이스 (Vector DB)

② SQLite를 내장하여 데이터 영속화(Persistence) 및 관리 기능을 기본 탑재한 완전한 형태의 DB

2) 장점

① 자동 영속화 및 컬렉션 관리: 디스크 저장 경로('persist_directory') 지정만으로 자동 저장되며, 테이블 개념의 컬렉션(Collection) 단위 관리 용이

② 풍부한 메타데이터 필터링: SQL의 'where' 조건절과 유사하게 특정 메타데이터(날짜, 출처, 카테고리 등) 기반의 정밀 필터링 가능

③ 뛰어난 확장성: 로컬 파일 모드뿐만 아니라 클라이언트-서버(REST API) 모드로의 확장 손쉬움

FAISS vs ChromaDB 핵심 차이 비교

- FAISS (Facebook AI Similarity Search)

3) 단점

- SQLite 및 DB 관리 레이어가 존재하여 대용량 임베딩 연산 시 FAISS보다 약간의 리소스 소모 존재

4) 핵심 사용법 (LangChain 기준)

① 벡터 저장소(vectorstore) 생성 및 자동 저장

: vectorstore = Chroma.from_documents(docs, embedding_model, persist_directory="./chroma_db")

② 검색 도구 생성 및 필터링

: retriever = vectorstore.as_retriever(search_kwargs={'filter': {'category': 'tech'}}

FAISS vs ChromaDB 핵심 차이 비교

- FAISS vs ChromaDB 비교 및 마이그레이션(Migration) 가이드

1) 핵심 비교 요약

항목	FAISS	ChromaDB
정체성	초고속 유사도 검색 라이브러리	완전한 형태의 Vector DB
데이터 저장 방식	In-Memory (수동 파일 저장)	디스크 자동 영속화 (SQLite 기반)
연산 성능	최상 (C++ / GPU 가속 지원)	우수 (일반 RAG 환경에 최적)
메타데이터 필터	수동 구현 및 제한적	기본 탑재 (where 필터 지원)
추천 활용 단계	파이프라인 수립 및 고속 검색 구현	서비스 영속화 및 DB 관리 확장

2) 실습 마이그레이션 (Migration) 핵심 포인트

① 벡터 저장소: 코드 '1줄' 변경만으로 교체 가능

② FAISS → ChromaDB 전환 시 코드 변경

- FAISS: vectorstore = FAISS.from_documents(docs, model)
- ChromaDB: vectorstore = Chroma.from_documents(docs, model, persist_directory="./chroma_db")

FAISS vs ChromaDB 핵심 차이 비교

- SQLite

1) 대부분의 컴퓨터에 내장된 데이터베이스

- 별도의 복잡한 서버 설치가 필요 없고 로컬에서 파일의 형태로 데이터베이스를 관리할 수 있음

2) sqlite3: 파이썬에서 SQLite를 사용할 수 있는 모듈

① 파이썬에 내장되어 있어서 별도의 설치 과정이 필요 없음

② 사용법 예시 코드

- 라이브러리 임포트

```
import sqlite3
```

- DB 생성 및 연결

```
conn = sqlite3.connect('test.db')
```

FAISS vs ChromaDB 핵심 차이 비교

- ChromaDB 메타데이터 필터링 예시

```
# 1. 사용자 질문 생성
user_question = "비밀번호 규정 알려줘"

# 2. 검색 도구 생성: 'filter' 파라미터를 통해 딕셔너리 형태로 메타데이터 필터링 조건 전달 → where category="보안" and year=2026
retriever_chroma = vectorstore_chroma.as_retriever(
    search_kwargs={
        'k': 1,
        'filter': {
            "$and":[
                {"category": {"$eq": "보안"}},
                {"year": {"$eq": 2026}}
            ]
        }
    }
)
# 3. 검색 도구를 사용하여 사용자 질문과 가장 유사한 문서(Document) 리스트 추출
retrieved_docs_chroma = retriever_chroma.invoke(user_question)
```

FAISS vs ChromaDB 핵심 차이 비교

- ChromaDB 연산자 정리

1) 비교 연산자 (Value 비교)

① $eq (Equal / =)

② $ne (Not Equal / !=)

③ $gt (Greater Than / >)

④ $gte (Greater Than or Equal / >=)

⑤ $lt (Less Than / <)

⑥ $lte (Less Than or Equal / <=)

2) 논리 연산자

① $and (AND)

② $or (OR)

FAISS vs ChromaDB 핵심 차이 비교

- MySQL 문법과 ChromaDB 메타데이터 필터링 비교

구분	MySQL 문법	랭체인 + ChromaDB 구현 코드
선택 조건	WHERE	'filter': { ... }
교집합 (그리고)	AND	"$and": [...]
조건 1 (카테고리)	category = '보안'	{"category": {"$eq": "보안"}}
조건 2 (연도)	year = 2026	{"year": {"$eq": 2026}}
출력 개수 제한	LIMIT 1	'k': 1

실습: ChromaDB + pdf 구현



실습: FAISS vs ChromaDB 핵심 차이 비교

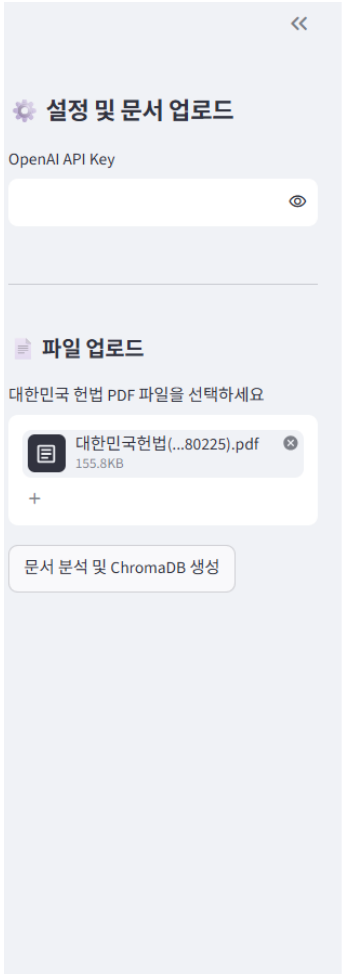
- 실습 결론

단순 유사도 계산은 FAISS가 빠르지만, 실무 환경처럼 '특정 조건+유사도'를 동시에 검색해야 하는 대용량 RAG 시스템에서는 ChromaDB가 구조적으로 훨씬 강력하다



실습: 구조화된 바이브 코딩을 이용 대화형 "대한민국 헌법 Q&A 챗봇" 구현

- 생성형 AI 입력 데이터
  1) 실습 파일
  2) 프롬프트





실습: 구조화된 바이브 코딩을 이용 대화형 “대한민국 헌법 Q&A 챗봇” 구현

25. 서울시 상권 분석 및 보고서 작성 멀티 에이전트 개발

시나리오(예시)

1. 서울시 상권 분석 및 보고서 작성 AI 에이전트 APP 개발 프로젝트

2. 가상 기업: (주)넥스트 프랜차이즈 파트너스

최신 F&B 트렌드를 선도하는 유망 스타트업의 브랜드 런칭 및 가맹 사업 확장 컨설팅 전문 기업

3. 현황 및 문제점

1) 직감에 의존하는 입지 선정: 신규 가맹점 출점 위치를 결정할 때, 컨설턴트의 개인적인 경험과 부정확한 유동 인구 정보에 크게 의존하여 실패 리스크가 높음
2) 느리고 피상적인 시장 조사: 특정 지역의 상권 분석 보고서를 작성하는 데 1~2주가 소요되며, "2024년 골목상권 유형의 상권들 중에서 30대 매출액 총합이 가장 높은 상위 3곳"과 같은 핵심 타겟 고객 분석이 빠르게 이루어지지 못함
3) 데이터 분석팀의 병목 현상: 모든 컨설턴트의 데이터 분석 요청이 소수의 분석팀에 몰려, 정작 중요한 '경쟁 브랜드 분석'이나 '미래 상권 예측'과 같은 핵심 전략 업무에 집중하지 못하고 있음

시나리오(예시)

4. 프로젝트 목표

1) 데이터 기반 입지 선정: 서울시의 실제 분기별, 연령별, 시간대별 매출 데이터를 기반으로 신규 가맹점의 성공 확률을 높이는 데이터 기반 의사결정 시스템을 구축
2) 시장 조사 시간 단축: 컨설턴트가 자연어 질문만으로 수 분 내에 원하는 상권에 대한 전문 분석 보고서를 얻을 수 있도록 하여, 업무 생산성을 향상

5. 시스템 아키텍처

1) 핵심: LangGraph와 PydanticAI Agent를 사용하여 '질문 재구성 → SQL 생성 → 실행 → 보고서 작성'의 4단계 하이브리드 워크플로우 설계
2) PydanticAI Agent: 각 노드를 구성하는 Agent(함수), LLM의 응답이 미리 정의된 Pydantic.BaseModel을 준수하도록 하여 데이터 일관성을 보장, 에이전트 간 데이터 전달의 안정성 강화
3) AI 모델: PydanticAI Agent 기반의 gpt-4o-mini 모델을 채택하여 강력한 추론과 구조화된 데이터 처리 구현
4) 데이터베이스: 구현하기 편리한 SQLite 데이터베이스 채택
5) 대화 관리: AsyncSqliteSaver를 통해 과거 대화 맥락을 기억하고 연속성 있는 분석 제공

시나리오(예시)

6. 단계별 개발 시나리오

1) 1단계: 데이터 파이프라인 구축

① 목표: AI가 정확한 NL2SQL을 수행할 수 있도록 고품질의 최신 상권 데이터를 확보하고 구조화

② 세부 과업: 서울 열린데이터광장 API를 활용한 데이터 수집 자동화

시나리오(예시)

6. 단계별 개발 시나리오

2) 2단계: 핵심 로직 개발 (Core Logic)

① 목표: PydanticAI Agent와 LangGraph를 결합한 4단계 지능형 분석 노드 구현

② 세부 과업

- 질문 재구성 노드 (Rephrase): 이전 대화 내역을 분석하여 "그럼 거기와 흑리단길 상권을 비교할 경우 2024년도 여성 매출액은 어때?"를 "2024년도 성수동카페거리와 흑리단길 상권의 여성 매출액을 비교해 주세요"로 구체화
- SQL 생성 노드 (NL2SQL): 재구성된 질문과 DB 스키마를 결합하여 실행 가능한 SQLite 쿼리 자동 생성
- SQL 실행 노드 (Execution): 생성된 쿼리를 asyncio.to_thread로 안전하게 실행하여 데이터 추출
- 보고서 생성 노드 (Report): 분석 결과와 사용자 의도를 결합하여 비즈니스 통찰력이 담긴 마크다운 보고서 작성

시나리오(예시)

6. 단계별 개발 시나리오

3) 3단계: 구조화된 바이브 코딩을 이용한 웹 APP 개발

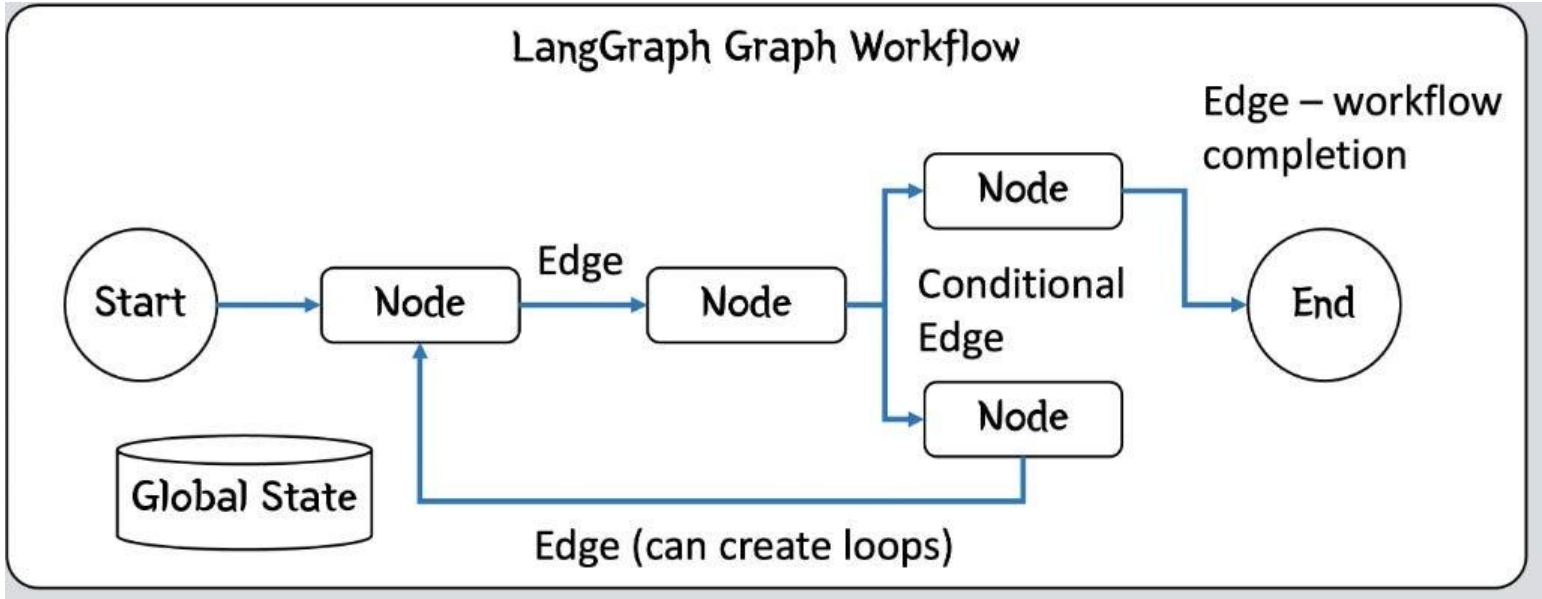
① 목표: 사용자가 AI 에이전트와 효율적으로 상호작용할 수 있는 웹 APP을 구현합니다

② 세부 과업: gemini와 같은 생성형 AI를 이용하여 개발한 코드와 개발 시나리오를 제시하고 로컬에서 작동하는 웹 APP을 개발한다.

25-1. LangGraph

LangGraph

- 정의: 복잡한 AI 에이전트를 만들기 위한 프레임워크로서, AI가 여러 단계를 거쳐 생각하고, 도구를 사용하며, 결정을 내리는 과정을 마치 '순서도'처럼 명확하게 설계하고 제어할 수 있게 도와준다.
- 그래프(Graph): 마치 '순서도'처럼 명확하게 설계한 '작업 흐름에 대한 설계도'



LangGraph

- 그래프(Graph)의 핵심 구성 요소

1) 노드 (Node): "누가 무엇을 하는가"

① 그래프의 각 지점, 보통 하나의 Python 함수나 에이전트로 정의

② 예: "검색 수행 노드", "답변 생성 노드", "결과 검토 노드"

2) 엣지 (Edge): "어디로 이동하는가"

① 노드와 노드를 연결하는 화살표로, 작업의 진행 방향을 나타냄

② 종류

- 일반 엣지: 하나의 작업이 끝나면 무조건 다음 작업으로 이동
- 조건부 엣지 (Conditional Edge): 조건에 따라 어떤 노드로 갈지 결정하는 갈림길 역할을 수행
  (예: 웹 검색 결과가 있으면 '요약하기' 노드로, 없으면 '다른 키워드로 검색하기' 노드로 이동)

LangGraph

- 그래프(Graph)의 핵심 구성 요소

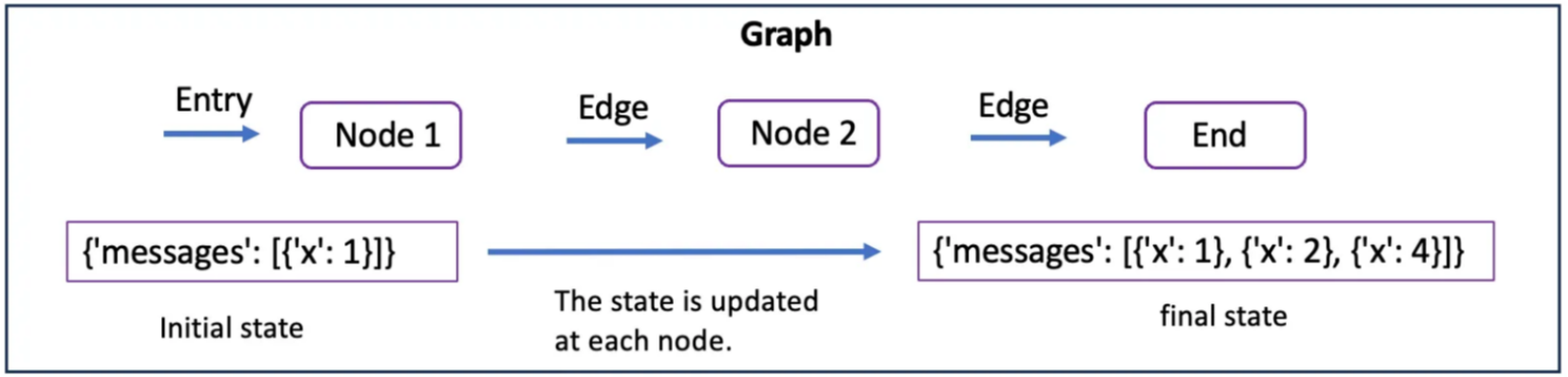
3) 상태 (State): 그래프의 모든 노드(Node)가 공유하고 각 노드의 실행 결과를 저장하는 '공동 작업 화이트보드(공유 메모리)'

① 무엇을 담나요?

- 사용자의 초기 질문, 각 노드의 작업 결과, 중간 계산 값 등 작업에 필요한 모든 정보를 기록하고 업데이트

② 왜 중요한가요?

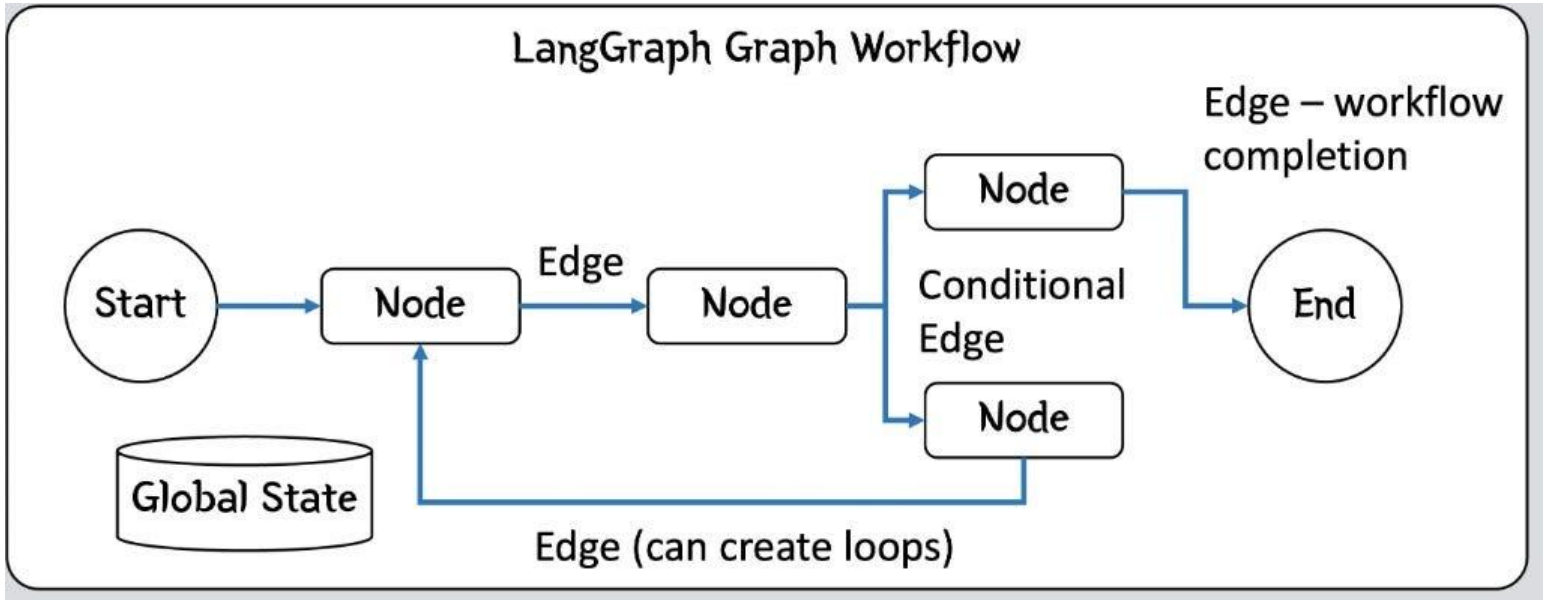
- 에이전트가 과거의 행동을 기억하고, 기억을 바탕으로 복잡한 결정을 내리며, 사용자와 상호작용하는 과정에서 일관성을 유지하게 한다.



LangGraph

- 목표

1) 한눈에 파악: 복잡한 작업 흐름을 명확하게 설계하고 관리한다.
2) 정보 공유: 작업 과정에서 중요한 정보를 '상태(State)' 라는 곳에 저장하여 장기적인 대화 또는 복잡한 작업 흐름에서 AI가 정보와 맥락을 잃지 않도록 한다.
3) 유연한 대응: 특정 조건에 따라 AI가 특정 작업을 반복하거나 다른 경로로 전환할 수 있도록 한다.



LangGraph의 핵심 구성 요소: 상태(State)

- 상태(State) 정의하기: '공동 작업 화이트보드'에 규칙 만들기

우리의 상태(State)는 여러 작업자들이 함께 쓰는 '공동 작업 화이트보드' 와 같습니다.
그런데 여러 사람이 아무 규칙 없이 보드를 사용하면 금방 어지러워지겠죠? 숫자를 적어야 할 칸에 그림을 그리거나, 중요한 메모를 지워버리는 실수가 생길 수 있습니다.

AI도 마찬가지입니다. 상태에 데이터가 뒤죽박죽 섞이면 AI는 오작동을 일으킵니다.
따라서 우리는 이 '공동 작업 화이트보드'에 명확한 규칙을 만들어야 합니다.
이때 사용하는 강력한 도구가 바로 typing 모듈과 Pydantic 라이브러리입니다.

LangGraph의 핵심 구성 요소: 상태(State)

- 상태(State) 정의하기: '공동 작업 화이트보드'에 규칙 만들기

1) typing 모듈

① 타입 힌트 (Type Hint) 기능 제공

- Type Hint: 노드의 매개변수에 "이 안에는 어떤 종류의 데이터만 들어올 수 있어!" 라고 알려주는 기능
- 예시: def agent_node(state): ⇒ def agent_node(state: str):

② 왜 사용하나요?

- 코드를 읽는 사람이 "아하, 이 변수에는 문자열이 들어가는구나!" 하고 쉽게 이해할 수 있다.
- 실수로 문자열이 들어갈 자리에 숫자를 넣으려고 하면, "코딩 프로그램" 또는 "코드 편집 프로그램"이 "이거 잘못된 것 같은데요?" 하고 미리 알려줘서 버그를 예방할 수 있다.

LangGraph의 핵심 구성 요소: 상태(State)

- 상태(State) 정의하기: '공동 작업 화이트보드'에 규칙 만들기

1) typing 모듈

③ 예시 심화

- def agent_node(messages: list[str]):

  ⇒ "messages에는 글자들(str)이 담긴 리스트(List)만 넣어주세요."

- def agent_node(retries: int):

  ⇒ "retries에는 정수(int)만 넣어주세요."

- def agent_node(user_info: dict[str, int]):

  ⇒ "user_info에는 '이름'처럼 문자열(str)로 된 key와 '나이'처럼 숫자(int)로 된 값이 짝을 이루는 dict 데이터만 넣어주세요."

- def agent_node(user_name: str | None):

  ⇒ "user_name에는 문자열(str)를 넣거나, 아니면 비워둬도(None) 괜찮아요."

LangGraph의 핵심 구성 요소: 상태(State)

- 상태(State) 정의하기: '공동 작업 화이트보드'에 규칙 만들기

1) typing 모듈

④ TypedDict 제공

- 키(key)의 종류와 값(value)의 타입이 미리 정해진 dict 자료형을 만들 수 있는 설계도
- 일반 dict: "아무거나 자유롭게 적는 빈 노트"
- TypedDict: "이름, 나이, 주소처럼 정해진 항목(key)과 특정한 자료형의 값(value)만 채워야 하는 설문지"
- LangGraph: TypedDict를 사용해 상태(State)라는 '공동 작업 화이트보드'의 데이터 형식을 명확하게 정의

LangGraph의 핵심 구성 요소: 상태(State)

- 상태(State) 정의하기: '공동 작업 화이트보드'에 규칙 만들기

1) typing 모듈

⑤ TypedDict를 이용한 상태(State) 정의 예시

```
# 상태(State) 정의
class AgentState(TypedDict):
    """
    1. 첫 번째 항목(key): messages
    2. 사용자와 에이전트가 주고받은 모든 대화 기록(HumanMessage, AIMessage, ToolMessage)을 순서대로 저장합니다(덮어쓰기 X).
    3. 여기에는 [HumanMessage(content), AIMessage(content), ToolMessage(content) ...] 형태의 리스트만 입력할 수 있습니다.
    4. '값(value)'에 부가 정보를 추가할 수 있는 기능을 가진 typing 모듈의 Annotated와 리스트 자료형의 대화 기록을 병합(append)하여
       순서대로 저장할 수 있는 langgraph의 전용 함수 add_messages를 사용하여 '값'을 정의
    """
    messages: Annotated[list[BaseMessage], add_messages]

    ## 두 번째 항목(key): tool_output
    ## 여기에는 문자열이 들어오거나 혹은 비어있을(None) 수 있습니다.
    tool_output: str | None
```

LangGraph의 핵심 구성 요소: 상태(State)

- 상태(State) 정의하기: '공동 작업 화이트보드'에 규칙 만들기

1) typing 모듈

⑤ TypedDict를 이용한 상태(State) 정의 예시

```
# 노드(Node) 함수 정의
def call_llm(state: AgentState):
    ## 실행 내용: LLM이 도구 사용을 지시
    return {"messages":[AIMessage(content="도구 사용을 요청합니다.")]}

def call_tool(state: AgentState):
    ## 실행 내용: 도구 실행 및 결과 생성
    tool_result = "도구 실행 결과: 데이터 처리 완료."
    return {"messages": [ToolMessage(content=tool_result)],  "tool_output": tool_result}
```

LangGraph의 핵심 구성 요소: 상태(State)

- 상태(State) 정의하기: '공동 작업 화이트보드'에 규칙 만들기

  2) Pydantic 라이브러리

  ① 기능: '공동 작업 화이트보드'인 '상태(State)'의 기본적인 '형식'을 정의하는 것을 넘어 **유효성 검사**(입력된 데이터가 미리 정의한 형식과 조건에 맞는지 확인하여 맞지 않으면 즉시 에러를 발생시켜서, 잘못된 데이터가 시스템 깊숙이 침투하여 갑자기 프로그램이 멈추는 상황을 사전에 차단)**을 통해서 에이전트의 안정성 및 신뢰성 제공**

LangGraph의 핵심 구성 요소: 상태(State)

- 상태(State) 정의하기: '공동 작업 화이트보드'에 규칙 만들기

2) Pydantic 라이브러리

② 유효성 검증의 핵심 도구: BaseModel 자료형

	TypedDict 자료형	BaseModel 자료형
제공	typing 모듈 (파이썬 내장 모듈)	Pydantic 라이브러리(외부 라이브러리)
기능	상태(State)'의 기본적인 '형식'을 정의하는 설계도	상태(State)'의 기본적인 '형식'을 정의하는 설계도 + 강력한 검증(like 검문소)
실행 시 검증	• '상태'에 어떤 항목(key)과 어떤 자료형의 값이 있어야 하는지 힌트만 제시 • 실행 중에 잘못된 값이 들어와도 막지 못함 → 예방(X), 나중에 에러 발생	• 실행 중에 잘못된 값이 들어오면 즉시 에러 발생 → 예방이 목적
결과물 타입	파이썬 dict 자료형	Pydantic 자료형

LangGraph의 핵심 구성 요소: 상태(State)

- 상태(State) 정의하기: '공동 작업 화이트보드'에 규칙 만들기

2) Pydantic 라이브러리

③ Field() 함수

- 기능: 각 항목(필드)에 대해 상세하고 강력한 규칙 설정

>> 기본값 설정: '값'이 안 들어왔을 때 사용할 '기본값'을 정할 수 있다.

>> 제약 조건 설정: 필드에 입력되는 숫자 크기 제한, 문자열 길이 제한 등을 설정할 수 있다

>> 설명(Description) 추가: 해당 필드에 대한 설명을 추가하면, LLM에게 전달되는 시스템 프롬프트의 일부로 포함되어 LLM에게 각 필드의 명확한 의도를 설명하여 LLM이 해당 필드에 어떤 데이터를 채워야 할지 명확히 인지하게 되어 LLM의 정확도를 향상시킬 수 있다

LangGraph의 핵심 구성 요소: 상태(State)

- 상태(State) 정의하기: '공동 작업 화이트보드'에 규칙 만들기

2) Pydantic 라이브러리

④ BaseModel 자료형과 Field 함수로 상태(State) 정의(예시)

```python
# 상태(State) 정의
class AgentState(BaseModel):
    # TypedDict와 동일하게 Annotated와 add_messages를 사용하여 모든 대화 기록을 순서대로 저장
    messages: Annotated[list[BaseMessage], add_messages] = Field(default_factory=list)
    # 도구 함수 실행의 결과 저장
    tool_output: str | None = Field(default=None )

# 노드(Node) 함수 정의
def call_llm(state: AgentState):
    ## 실행 내용: LLM이 도구 사용을 지시
    return {"messages":[AIMessage(content="도구 사용을 요청합니다.")]}

def call_tool(state: AgentState):
    ## 실행 내용: 도구 실행 및 결과 생성
    tool_result = "도구 실행 결과: 데이터 처리 완료."
    return {"messages": [ToolMessage(content=tool_result)],  "tool_output": tool_result}
```

실습: typing 모듈의 TypedDict를 이용한 LangGraph 상태 정의

- 실습 개요
  1) 상태 정의
  2) 노드 함수 정의
  3) 그래프 구성 및 완성: 그래프 생성, 노드 추가, 시작점 설정, 엣지 추가, 컴파일
  4) 실행 및 확인

실습: typing 모듈의 TypedDict를 이용한 LangGraph 상태 정의



실습: Pydantic BaseModel을 이용한 LangGraph 상태 정의

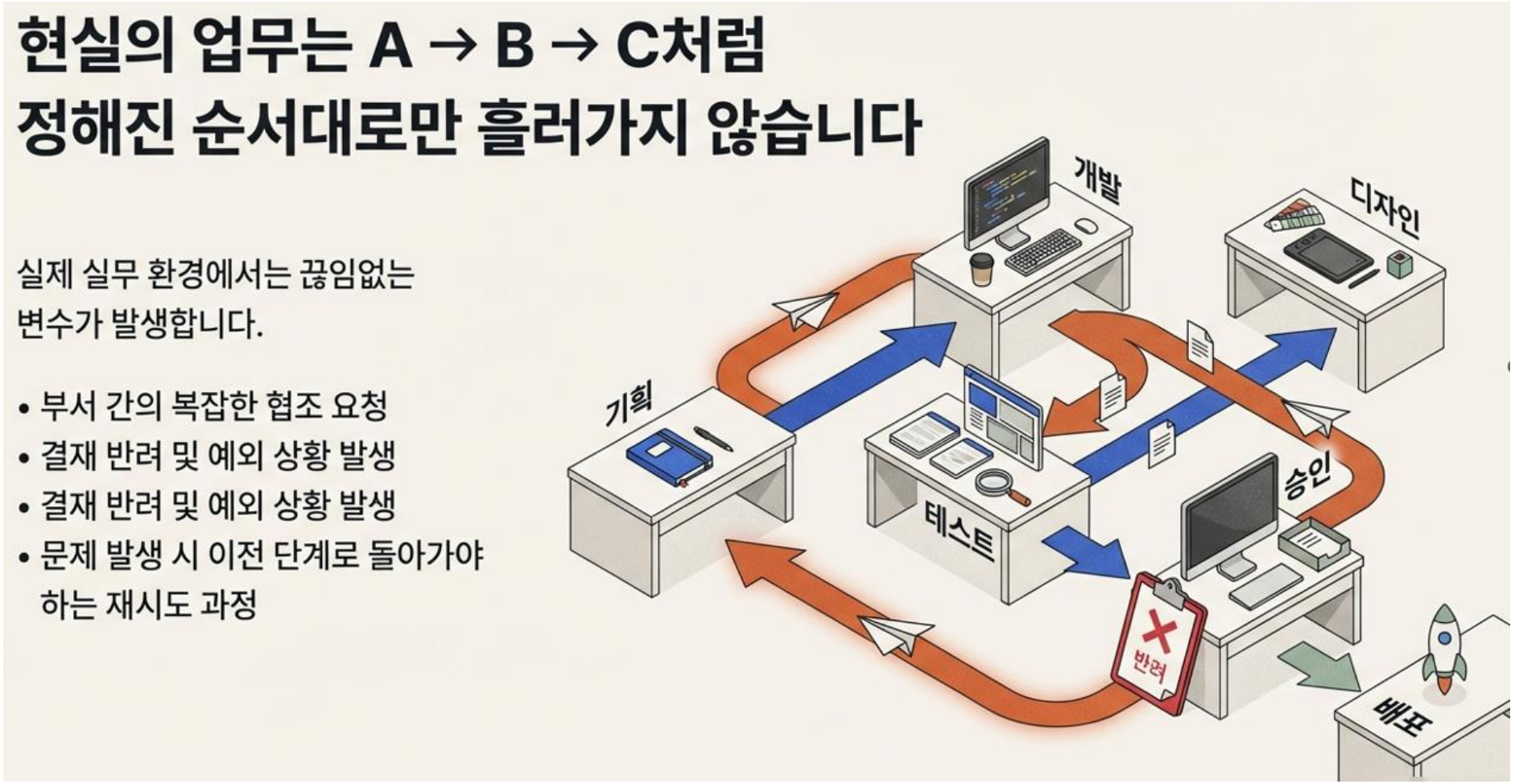
- 실습 개요
  1) 상태 정의
  2) 노드 함수 정의
  3) 그래프 구성 및 완성: 그래프 생성, 노드 추가, 시작점 설정, 엣지 추가, 컴파일
  4) 실행 및 확인

실습: Pydantic BaseModel을 이용한 LangGraph 상태 정의



LangGraph 에이전트

- 조건부 엣지



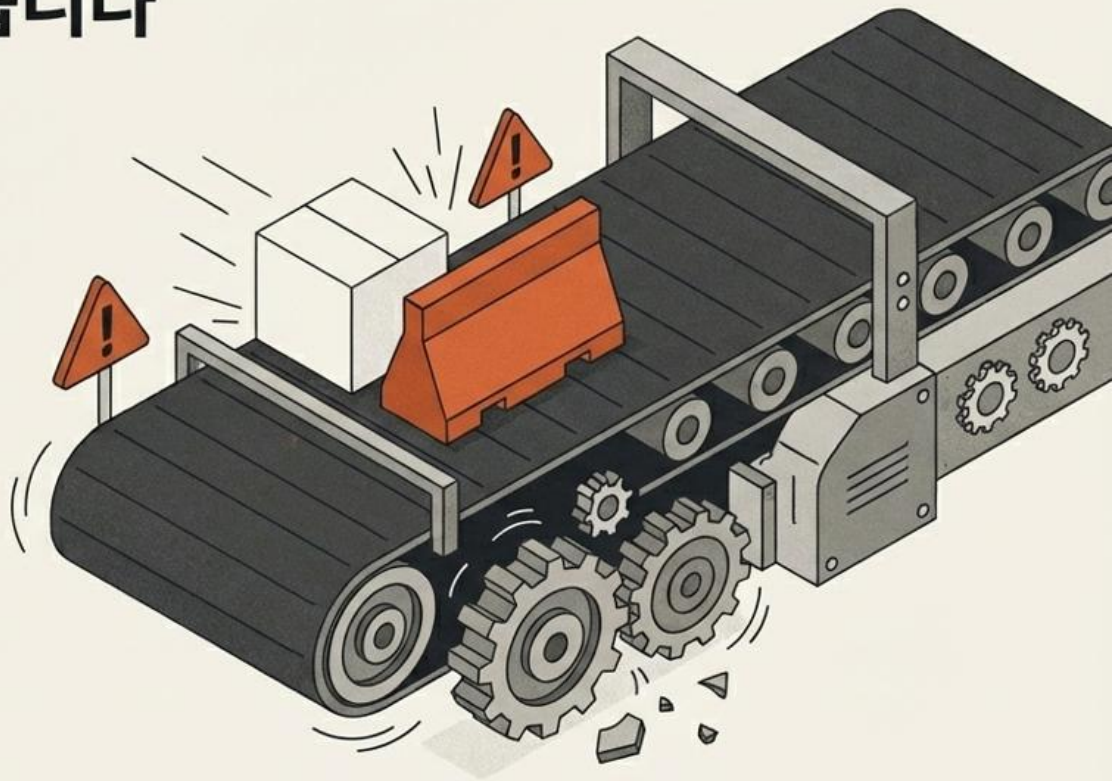
LangGraph 에이전트

- 조건부 엣지

기존의 AI 워크플로우는 경직된 '컨베이어 벨트'와 같았습니다

기존의 체인(Chain) 방식은
정해진 순서대로만 작동합니다.

중간에 예외 상황이 발생하거나
하나의 작업이 실패하면
전체 과정이 멈춰버리는 한계를
지니고 있었습니다.



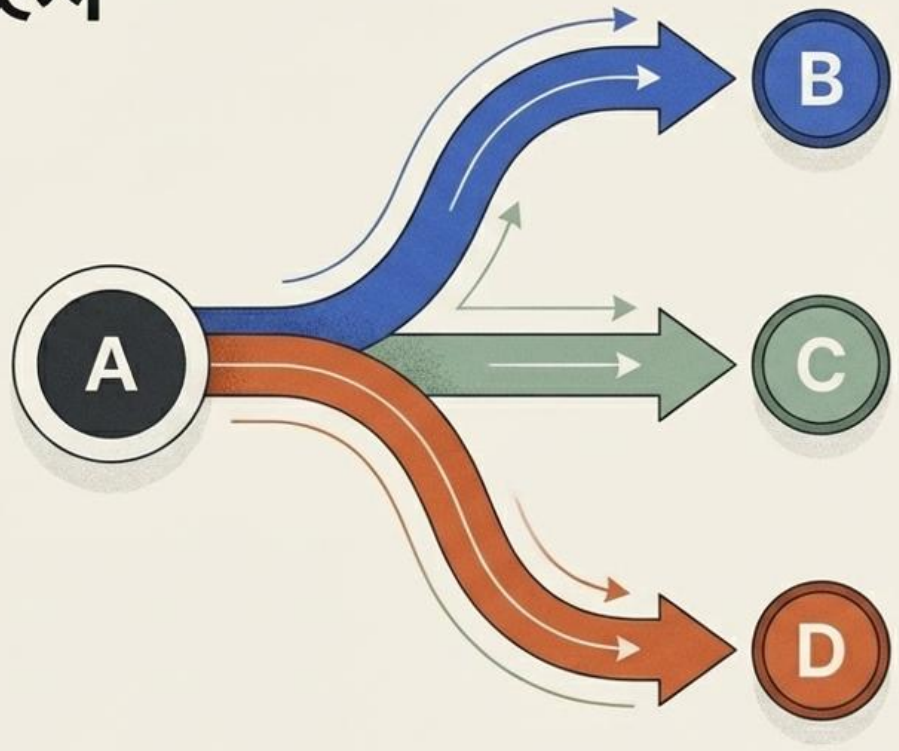
LangGraph 에이전트

- 조건부 엣지

패러다임의 전환: 상황에 따라 스스로 길을 찾는 '조건부 엣지'

LangGraph의 진정한 힘은 '조건부 엣지(Conditional Edge)'에서 나옵니다.

특정 함수의 실행 결과에 따라 A 노드 다음에 B로 갈지, C로 갈지, D로 갈지를 동적으로 결정합니다.
(업계에서는 이를 '조건부 라우팅' 또는 '조건부 분기'라고도 부릅니다.)



LangGraph 에이전트

- 조건부 엣지

복잡한 논리도 3단계 매핑 구조로 명확하게 제어됩니다

add_conditional_edges() 메서드를 사용해
다음 3가지 핵심 요소를 연결합니다.

[1단계] 어디서 출발할지
(시작 노드)

[2단계] 어떤 기준으로 판단할지
(조건 함수)

[3단계] 결과에 따라 어디로 보낼지
(경로 맵핑)

```
workflow.add_conditional_edges(
    'action',          # 1단계: 시작 노드
    should_continue, # 2단계: 조건을 판단
    판단하는 함수
    {
        'continue': 'action', # 3단계:
        결과가 continue이면 action으로 이동
        'end': END   # 3단계: 결과가 end이
    }   면 종료
)
```

실습: LangGraph 조건부 분기를 활용한 동적 에이전트 워크플로우 설계

- 실습 개요
  1) API 키 로드
  2) 그래프 상태 정의: TypedDict 사용
  3) LLM 설정
  4) 노드 함수 정의
  5) 조건 판단 (조건부 라우팅) 함수 정의
  6) 그래프 구성 및 완성: 그래프 생성, 노드 추가, 시작점 설정, 엣지 추가, 컴파일
  7) 그래프 실행 및 조건부 분기 흐름 관찰

실습: LangGraph 조건부 분기를 활용한 동적 에이전트 워크플로우 설계



실습: 고객 응대 Chatbot Agent 구현

- 실습 개요
  - 목표

    : A → B → C처럼 정해진 순서대로만 작동하는 선형적인(Linear) RAG 체인을, 더욱 강력하고 확장 가능한 LangGraph의 '노드(Node)'로 포장하여, 앞으로 만들 더 복잡한 시스템의 '부품'으로 재사용할 수 있도록 업그레이드

실습: 고객 응대 Chatbot Agent 구현

- 실습 개요
  - 과정
    1) API 키 로드
    2) RAG 시스템 기반 설정: 데이터 로딩, 텍스트 분할, 임베딩 및 벡터 저장소 구축, LLM 설정
    3) RAG 체인 구성: LangChain Expression Language(LCEL)로 구성
    4) 그래프 상태 정의: pydantic 라이브러리의 BaseModel 자료형 사용
    5) 노드 함수 정의
    6) 그래프 구성 및 완성: 그래프 생성, 노드 추가, 시작점 설정, 엣지 추가, 컴파일
    7) 에이전트 실행

실습: 고객 응대 Chatbot Agent 구현



LangGraph 에이전트

- 에이전트 유형: 싱글 에이전트 (Single Agent), 멀티 에이전트 (Multi-Agent)
- 에이전트 유형별 비교

1) LangGraph는 싱글 에이전트뿐만 아니라, 여러 에이전트가 상호작용하는 복잡한 시스템까지 효율적으로 구현할 수 있는 유연성을 제공

2) 싱글 에이전트 (Single Agent)

① 개념: 하나의 LLM 에이전트가 정의된 모든 도구(Tool)와 자신의 추론 능력을 활용하여 처음부터 끝까지 하나의 복잡한 작업을 완료

② LangGraph의 그래프, 상태(state), 조건부 로직을 사용하여 에이전트의 내부 추론-행동 루프를 상세하게 정의하고 제어할 수 있음

③ 주요 활용 시나리오

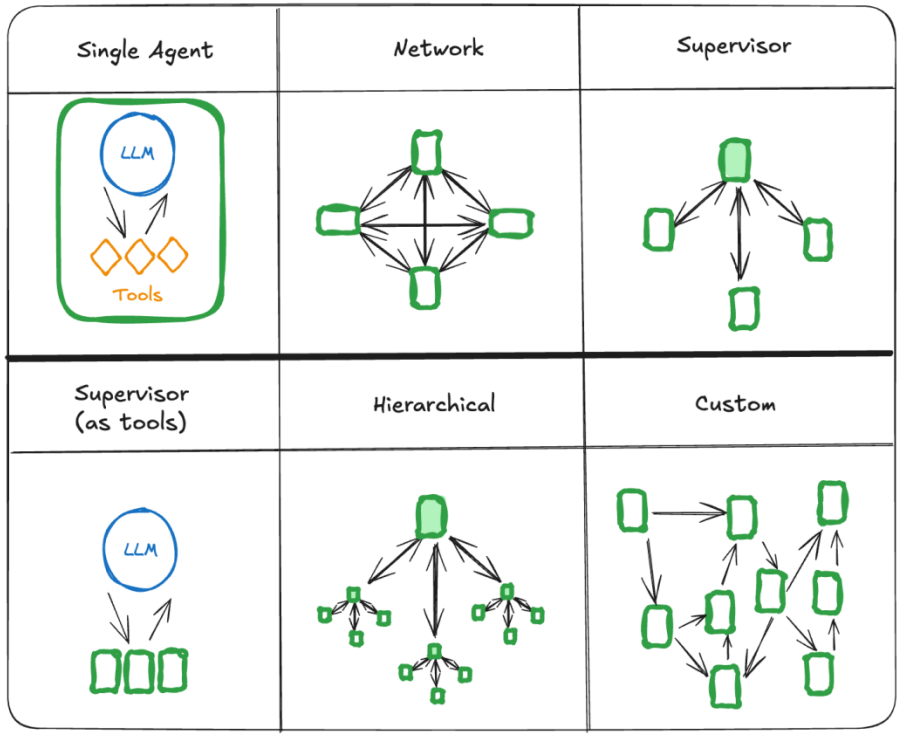
- 질의응답 시스템
- 웹 검색을 통한 정보 수집 및 보고서 초안 작성
- 데이터 분석 에이전트 등

LangGraph 에이전트

- 에이전트 유형별 비교

3) 멀티 에이전트 (Multi-Agent) 시스템

① 개념: 여러 개의 독립적이거나 상호 연결된 에이전트가 각자의 역할을 수행하며 협력하거나 경쟁하여 하나의 복잡한 목표를 달성하는 시스템. 각 에이전트는 특정 전문 분야나 역할을 가짐



LangGraph 에이전트

- 에이전트 유형별 비교

3) 멀티 에이전트 (Multi-Agent) 시스템

② LangGraph의 그래프, 상태(state), 조건부 로직을 사용하여 에이전트의 내부 추론-행동 루프를 상세하게 정의하고 제어할 수 있음

- 에이전트 간 협업 조율: LangGraph의 노드를 각 에이전트의 '결정 지점'이나 '액션 실행 지점'으로 활용하여 에이전트 간의 정보 전달과 제어 흐름을 명확하게 정의할 수 있음
- 공유 상태를 통한 정보 교환: 모든 에이전트가 공유하는 '상태(state)' 를 통해 필요한 정보를 교환하고, 다른 에이전트가 생성한 결과를 활용할 수 있음

LangGraph 에이전트

- 에이전트 유형별 비교

3) 멀티 에이전트 (Multi-Agent) 시스템

③ 주요 활용 시나리오

- 복합 고객 서비스 처리 시스템: 사용자 문의를 자동으로 분류하는 '분류 에이전트', 기술 관련 문제를 해결하는 '기술 지원 에이전트', 환불 절차를 안내하는 '환불 에이전트' 등이 협력하여 고객 요청을 효율적으로 응대하고 해결하는 시스템
- 자동화된 프로젝트 기획 및 개발 워크플로우: 아이디어를 구상하고 계획을 수립하는 '기획 에이전트', 실제 코드를 작성하고 기능을 구현하는 '개발 에이전트', 그리고 코드의 품질과 기능을 검증하는 '테스트 에이전트' 등이 연계하여 프로젝트 진행을 자동화하는 워크플로우

LangGraph 에이전트

- 에이전트 유형별 비교

4) 싱글 에이전트와 멀티 에이전트 시스템 비교 정리

: LangGraph는 두 가지 유형의 에이전트 시스템 모두를 지원하지만, 각 아키텍처는 고유한 장단점이 존재

특징	싱글 에이전트 시스템	멀티 에이전트 시스템
복잡성	상대적으로 단순함	에이전트 간 상호작용 및 조율로 인해 복잡도가 높음
역할 분담	모든 작업을 하나의 에이전트가 처리	각 에이전트가 전문화된 역할을 수행
확장성	에이전트의 능력에 따라 제한적일 수 있음	새로운 에이전트 추가로 기능 확장 용이, 병렬 처리 유리
오류 처리	단일 실패 지점 집중	한 에이전트의 실패가 전체 시스템에 미칠 영향 고려 필요 (분산)
성능	순차적 처리 위주, 최적화에 따라 달라짐	병렬 처리를 통해 성능 향상 가능, 오버헤드 발생 가능
관리 용이성	설정 및 디버깅이 비교적 간단함	에이전트 간 통신, 동기화, 충돌 해결 등 복잡한 관리 필요
적합한 시나리오	명확하고 범위가 한정된 복합 작업	복잡하고 다양한 하위 작업으로 나눌 수 있는 대규모 문제 해결

LangGraph 에이전트

- LangGraph의 고급 기능

1) 비동기 실행 (Asynchronous Execution)

① 개념

- 대부분의 LLM 애플리케이션은 네트워크 호출(LLM API 호출, 외부 웹 서비스 접근, 데이터베이스 조회 등)과 같이 '네트워크 I/O 바운드(I/O-bound)'작업에 크게 의존하게 됨
- 네트워크 I/O 바운드 작업: 주로 외부 리소스와의 통신에 의해 성능이 좌우되는 작업, 외부 시스템과의 통신에 많은 시간을 할애하게 됨
- 네트워크 I/O 바운드 작업의 예시
  - ▶ 외부 API 호출: LLM(Large Language Model) 호출, 검색 엔진 API 호출
  - ▶ 데이터베이스 작업: 데이터베이스 조회 , 데이터 읽기/쓰기
  - ▶ 파일 시스템 작업: 파일 읽기/쓰기

LangGraph 에이전트

- LangGraph의 고급 기능

1) 비동기 실행 (Asynchronous Execution)

① 개념:

- 동기(Synchronous) 방식으로 이러한 작업을 처리하면, 하나의 I/O 작업이 완료될 때까지 전체 프로그램이 멈춰서 다음 작업을 기다려야 하며 이는 시스템의 효율성을 크게 저하시킴

**“요리사가 물을 끓이기 시작한 뒤, 물이 다 끓을 때까지 가만히 서서 기다리다가,
다 끓으면 다음 재료 손질을 한다”**

- 비동기(Asynchronous) 실행은 이러한 문제를 해결하기 위한 프로그래밍 방식으로 특정 작업(특히 I/O 작업)이 완료될 때까지 기다리지 않고, 해당 작업이 진행되는 동안 다른 작업을 실행할 수 있도록 제어하는 방식

“요리사가 물을 올리고 끓기를 기다리는 동안 재료 손질을 한다”

② 구현: asyncio 라이브러리를 통해 비동기 프로그래밍을 구현

LangGraph 에이전트

- LangGraph의 고급 기능

1) 비동기 실행 (Asynchronous Execution)

③ 비동기 실행의 핵심 개념: 코루틴(Coroutine)

- 개념: 일반 함수와 달리 함수 호출 시 즉시 실행되지 않고 실행이 일시 중지되었다가 나중에 다시 시작할 수 있는 특별한 종류의 함수
- async, await 예약어(키워드)를 이용해서 정의 및 실행
- async def: 함수를 코루틴 함수로 만드는 키워드
- await:

▶코루틴 함수 내부에서 다른 코루틴 함수(주로 시간이 걸리는 I/O 작업)의 완료를 기다릴 때 사용

▶await 대상 함수가 완료될 때까지 해당 코루틴 함수의 실행이 중단되며, 다른 작업을 실행할 수 있음

▶await 대상 함수가 완료되면, 중단되었던 코루틴 함수 재실행

LangGraph 에이전트

- LangGraph의 고급 기능

1) 비동기 실행 (Asynchronous Execution)

④ 코루틴(Coroutine) 예시

```
async def cook_ramen():
    print("라면 끓이기 시작!")
    # await 키워드는 잠시 멈추라는 신호이고 asyncio.sleep(1)은 1초(물이 끓는) 동안 기다리라는 뜻인데,
    # 이 동안 cook_ramen 함수는 잠시 멈추고 다른 작업을 할 수 있음
    await asyncio.sleep(1)
    print("면 넣기!")
    # 면 익히는 시간 2초 동안 대기, 이 동안 cook_ramen 함수는 잠시 멈추고 다른 작업을 할 수 있음
    await asyncio.sleep(2)
    print("라면 완성!")
    return "맛있는 라면"
```

∴ print("라면 끓이기 시작") → 1초 경과 → print("면 넣기!") -> 2초 경과 → print("라면 완성!") → 결과값: "맛있는 라면" 반환

⑤ 효과: I/O 대기 시간에 다른 작업을 동시에 처리(병렬 처리)하면 전체 작업을 효율적으로 처리할 수 있음

"요리사가 물을 올리고 끓기를 기다리는 동안 재료 손질을 할 수 있다"

LangGraph 에이전트: 고급 기능 적용 실습

- 비동기로의 변화 3단계

1) 1단계: I/O 호출부의 변화 (invoke ⇒ await ... ainvoke)

① 기존 '동기' 방식: retriever.invoke()와 rag_chain.invoke()는 응답이 올 때까지 프로그램 전체를 완전히 멈춰 세운다(Blocking).

② 변경 '비동기' 방식: 비동기 전용 함수인 ainvoke()가 대체. "이 작업은 시간이 걸리니 응답이 올 때까지 다른 일을 하고 있어라"고 컴퓨터(시스템)에게 제어권을 넘겨주기 위해 앞에 await 키워드를 반드시 붙인다.

```
# 기존 동기 방식
retrieved_docs = retriever.invoke(question)
response = rag_chain.invoke({"context": ..., "question": ...})

# ➔ 1단계 비동기 변환
retrieved_docs = await retriever.ainvoke(question)
response = await rag_chain.ainvoke({"context": ..., "question": ...})
```

LangGraph 에이전트: 고급 기능 적용 실습

- 비동기로의 변화 3단계

2) 2단계: 에이전트 노드 함수의 변화 (def ⇒ async def)

① 1단계에서 함수 내부에 await라는 키워드가 사용되면서, 이 함수를 감싸고 있는 '노드 함수'에도 변화가 일어난다. 파이썬 문법상 await 키워드는 반드시 async def로 선언된 함수 내부에서만 사용할 수 있다

② 기존 '동기' 방식: 일반 함수인 def rag_node(state):로 선언

③ 변경 '비동기' 방식: 함수 선언 예약어 def앞에 async를 붙여 async def rag_node(state):로 업그레이드. 이제 해당 노드는 LangGraph 안에서 비동기 전용 노드로 작동

```python
# 기존 동기 방식
def rag_node(state: AgentState):
    # 내부 일반 연산...

# ➔ 2단계 비동기 변환
async def rag_node(state: AgentState):
    # 내부에서 await 연산 가능!
```

LangGraph 에이전트: 고급 기능 적용 실습

- 비동기로의 변화 3단계

3) 3단계: 에이전트 실행 방식의 변화

: 노드가 비동기로 바뀌었으니, 이 노드들을 엮어서 만든 최종 결과물인 에이전트(agent)와 사용자의 입력을 받는 반복문(while True)도 동기 방식으로 실행할 수 없으며 비동기 방식으로 교체

```
# 기존 동기 방식 (코랩 셀일 실행)
inputs = agent.invoke(inputs)

# → 3단계 비동기 변환 (.py 파일 실행)
async def main():
    ...
    inputs = await agent.ainvoke(inputs) # 에이전트 비동기 구동

if __name__ == "__main__":
    asyncio.run(main()) # 비동기 엔진 시동
```

LangGraph 에이전트: 고급 기능 적용 실습

- 고객 응대 Chatbot Agent 업그레이드(1)



LangGraph 에이전트

- LangGraph의 고급 기능

  2) 체크포인팅(Checkpointing)

지금까지 만든 '상태' 기반 AI 에이전트는 대화가 끝나면 모든 것을 잊어버리는
휘발성 단기 기억만 가지고 있습니다.

하지만 실제 챗봇처럼 사용자와 '대화방(Thread)'을 유지하며
이전 대화를 기억하고 이어서 작업을 수행하게 하려면 어떻게 해야 할까요?

바로 이 '기억 지속성'을 가능하게 하는 핵심 기능이 '체크포인팅'입니다.

LangGraph 에이전트

- LangGraph의 고급 기능

2) 체크포인팅(Checkpointing)

① 용어 정리

- LangGraph의 '상태'는 에이전트 워크플로우의 모든 정보를 담고 있는 공유 메모리
- 체크포인트 (Checkpoint): 특정 노드가 실행된 직후의 업데이트된 상태(State) 데이터 그 자체이며, '스냅샷'이라는 용어로도 표현한다. (결과물)
- 체크포인팅 (Checkpointing): 에이전트의 현재 상태를 '체크포인트'로 만들어 영구적으로 저장하고, 필요할 때 다시 불러와 워크플로우를 특정 지점부터 이어서 실행할 수 있도록 하는 전체 프로세스를 의미한다. (행위)
- 체크포인터 (Checkpointer): '체크포인팅'을 실제로 수행하는 '핵심 도구'로서, 상태를 어디에, 어떻게 저장하고 불러올지 모든 방법을 알고 있는 상태 관리 도구이다. (도구)

LangGraph 에이전트

- LangGraph의 고급 기능

2) 체크포인팅(Checkpointing)

② 체크포인팅의 강력한 활용성: 이러한 체크포인팅 기능은 다음과 같은 상황에서 매우 유용하게 사용된다.

- 장기 실행 워크플로우: 긴 대화나 복잡한 분석 작업 도중 시스템이 재시작되더라도, 마지막 체크포인트를 불러와 중단된 지점부터 자연스럽게 작업을 이어갈 수 있다.
- 오류 복구: 에이전트 실행 중 오류가 발생하면, 가장 마지막에 성공적으로 저장된 체크포인트로 되돌아가 작업을 다시 시도할 수 있다.
- 디버깅: 특정 시점의 체크포인트를 열어보면 그 당시의 상태(State)를 그대로 확인할 수 있어, 에이전트의 행동을 분석하고 문제를 진단하기 매우 용이하다.

LangGraph 에이전트

- LangGraph의 고급 기능

2) 체크포인팅(Checkpointing)

③ 실전 구현: AsyncSqliteSaver

- LangGraph는 다양한 데이터베이스에 상태를 저장할 수 있는 여러 체크포인터를 제공한다.
- 가장 기본적으로 사용해 볼 수 있는 SQLite 데이터베이스 기반의 AsyncSqliteSaver를 사용하여 실습한다.
- 필요 패키지 설치
  - ▶ langgraph-checkpoint-sqlite: SQLite를 체크포인트 저장소로 사용하기 위한 라이브러리
  - ▶ aiosqlite: 비동기 방식으로 SQLite DB를 사용하기 위한 라이브러리
- AsyncSqliteSaver란?
  - ▶ 체크포인터(Checkpointer), 즉 '저장/불러오기 기능'을 담당하는 핵심 도구
  - ▶ 우리가 만든 그래프에 연결하면, 각 노드가 실행될 때마다 알아서 '업데이트된 상태'를 SQLite DB에 자동으로 저장하고 관리해준다.

LangGraph 에이전트

- LangGraph의 고급 기능

2) 체크포인팅(Checkpointing)

④ AsyncSqliteSaver 구현 예시

```python
async def main():
    # 체크포인트를 저장할 SQLite 데이터베이스 파일명 설정
    db_file = "checkpoints.sqlite"

    # 'async with'를 사용하여 AsyncSqliteSaver 객체 생성
    async with AsyncSqliteSaver.from_conn_string(db_file) as memory:
        # 'async with' 블록 안에서 그래프를 컴파일하고 실행해야 함
        graph = builder.compile(checkpointer=memory)
```

- async with 구문을 사용하여 AsyncSqliteSaver 체크포인터를 생성하고 DB 파일을 연결한다.
- async with를 사용하면 블록이 끝날 때 DB 연결이 자동으로 안전하게 닫힌다.
- 그래프를 컴파일할 때, checkpointer 매개변수에 위에서 만든 memory (DB에 연결된 체크포인터)를 전달한다.
- 그래프는 모든 대화 기록을 자동으로 저장하고 불러오는 '기억 지속성' 능력을 갖추게 된다.

LangGraph 에이전트: 고급 기능 적용 실습

- 체크 포인팅 기능 구현 시 핵심 변화 포인트 3가지

1) 메인 루프: 대화 기록의 보따리를 전달하는 방식 → 방 번호만 전달하는 방식 (가장 중요)

① 기존 방식: inputs라는 딕셔너리의 리스트에 append를 쓰며 과거 메시지를 계속 누적하여 에이전트에 전체 대화 기록을 넘겨주어야 한다.

② 변경: 방금 사용자가 입력한 새로운 질문 단 한 개만 initial_state에 입력한다. 대신, 이 대화방의 고유 식별자인 config({"configurable": {"thread_id": "user-7777"}})를 함께 전달한다. LangGraph가 이 thread_id를 보고 DB에서 과거 기록을 알아서 복원해 자동으로 새 질문과 병합(자동 누적)하여 에이전트로 전달한다.

⇒ 정리: thread_id → DB 조회 → 과거 상태 복원 → 새 질문과 병합(자동 누적) → 에이전트 노드로 전달

```
# 1단계: 과거 기록을 코드가 직접 무겁게 들고 다님
inputs["messages"].append(HumanMessage(content=user_question))
inputs = await agent.ainvoke(inputs)

# → 2단계 변환: 새 질문 하나와 '방 번호표(thread_id)'만 가볍게 전달
initial_state = {"messages": [HumanMessage(content=user_question)]}
final_state = await agent.ainvoke(initial_state, config=config)
```

LangGraph 에이전트: 고급 기능 적용 실습

- 체크 포인팅 기능 구현 시 핵심 변화 포인트 3가지

2) 프롬프트와 노드: 과거 대화 기록이 들어올 '전달 경로(Pathway)' 개설

① DB가 기억을 복원해 주더라도, 그 기억을 프롬프트를 통해 LLM에게 밀어 넣어주지 않으면 LLM은 과거 내용을 알 수 없다. 그래서 프롬프트와 노드 내부 구조에 과거 대화록 전달 경로(Pathway)를 만들어야 한다.

② 기존 방식: 프롬프트에 context와 question만 들어갔고, 노드에서도 현재 질문만 처리

③ 변경: MessagesPlaceholder를 이용하여 프롬프트 중간에 과거 기록이 들어갈 자리를 만든다. 그리고 노드 함수 내부에서 전체 메시지(state.messages) 중 마지막 질문을 제외한 모든 과거 내역을 쪼개어 rag_chain에 넘겨준다.

```
# 1단계: 현재 질문만 LLM에 전달
prompt = ChatPromptTemplate.from_messages([
    ("system", system_prompt),
    ("human", "컨텍스트:\n{context}\n\n질문:\n{question}")
])

# → 2단계 변환: 과거 기록(chat_history)이 들어갈 주입구 개설
prompt = ChatPromptTemplate.from_messages([
    ("system", system_prompt),
    MessagesPlaceholder(variable_name="chat_history"), # 뚫어놓은 주입구!
    ("human", "컨텍스트:\n{context}\n\n질문:\n{question}")
])
```

LangGraph 에이전트: 고급 기능 적용 실습

- 체크 포인팅 기능 구현 시 핵심 변화 포인트 3가지

3) 컴파일 시점: 에이전트에 '외부 기억 장치(DB)' 장착

① 에이전트의 그래프를 컴파일하는 시점에 하드디스크 역할을 할 데이터베이스와 융합시킨다.

② 기존 방식: 메모리가 필요 없으므로 그냥 workflow.compile()로 끝났음

③ 변경: 모든 '상태(state)'를 비동기 방식으로 영구 저장하는 체크포인터를 async with 구문으로 SQLite와 연결한 뒤, 컴파일 함수 내부 파라미터 'checkpointer'에 전달하여 에이전트에 외부 기억 장치(DB)를 융합시킨다.

```
# 1단계: 기억 장치 없는 에이전트 빌드
agent = workflow.compile()

# → 2단계 변환: DB 기억 장치를 장착하며 에이전트 빌드
async with AsyncSqliteSaver.from_conn_string(db_file) as memory:
    agent = workflow.compile(checkpointer=memory)
```

LangGraph 에이전트: 고급 기능 적용 실습

- 고객 응대 Chatbot Agent 업그레이드(2)



LangGraph 에이전트: 멀티 에이전트 적용 실습

- 멀티 에이전트 적용 시 핵심 변화 포인트 4가지

1) 노드의 세분화: 1개의 독점 노드 → 3개의 전문 노드(에이전트)로 분할

① 기존 rag_node 하나가 하던 다양한 일들을 정확하게 3개의 async def 독립 노드로 쪼개어 분업화 달성

② rephrase_node (질문 재구성): 대화 기록을 파악해 "그거 전액 다 받으려면 조건이 뭐야?"를 "수강 신청한 강의를 전액 환불 받으려면 조건이 뭐야?"로 명확하게 바꾼 뒤 '상태'의 'rephrased_query' 필드에 담는다.

③ retriever_node (문서 검색): '상태'에 담긴 rephrased_query를 꺼내어 벡터저장소(FAISS DB)를 검색하고, 찾은 매뉴얼 본문을 '상태'의 'context' 필드에 담는다.

④ answer_node (답변 생성): 'context' 필드에 담긴 매뉴얼 본문과 원본 질문을 조합하여 최종적으로 고객에게 줄 정중한 답변 메시지를 생성한다.

LangGraph 에이전트: 멀티 에이전트 적용 실습

- 멀티 에이전트 적용 시 핵심 변화 포인트 4가지

2) 그래프 상태(AgentState)의 확장

① 기존 방식: messages 필드 1개

② 변경: 질문 재구성 에이전트가 만든 정제된 질문을 문서 검색 에이전트에게 전달해야 하고, 문서 검색 에이전트가 찾은 매뉴얼 내용을 답변 생성 에이전트에게 전달해야 한다. 이를 위해 AgentState에 정보를 공유할 필드를 추가해야 한다.

```
class AgentState(BaseModel):
    messages: Annotated[List[BaseMessage], operator.add] = Field(default_factory=list)
    rephrased_query: str = "" # 재구성된 질문 저장
    context: str = ""         # 검색된 매뉴얼 내용 저장
    next: str = ""            # 라우팅 정보
```

LangGraph 에이전트: 멀티 에이전트 적용 실습

- 멀티 에이전트 적용 시 핵심 변화 포인트 4가지

3) 흐름 제어의 진화: 직선 구조 → 조건부 분기 (Conditional Edges) 적용

① 노드가 여러 개로 늘어났기 때문에, 파이썬에게 "1번 다음엔 2번으로 가고, 2번 다음엔 3번으로 가라"고 교통정리를 해주어야 한다.

② 기존 방식: 시작하자마자 rag_node로 가고, 끝나면 바로 END로 빠지는 단순한 직선 구조

③ 변경: 각 노드가 일을 마칠 때마다 return {"next": "다음 노드 이름"} 형태로 '상태'에 행선지를 기록하면, 중앙 제어 장치인 라우터 함수(router)가 그 기록을 읽어 다음 에이전트를 연결하는 유연한 흐름 제어 시스템 도입

```python
# 상태창에 적힌 행선지를 읽어주는 네비게이션 함수
def router(state: AgentState) -> str:
    return state.next

# 흐름 조립
workflow.add_conditional_edges("rephrase", router) # rephrase가 지정한 next 노드로 이동
```

LangGraph 에이전트: 멀티 에이전트 적용 실습

- 멀티 에이전트 적용 시 핵심 변화 포인트 4가지

4) 체인(Chain)의 다원화: 1개 → 태스크별 전용 체인 배치

① 뇌(LLM)를 하나만 쓰던 방식에서, 두 개의 목적을 가진 전용 브레인 체인으로 분리하여 각자의 프롬프트 부여

② rephrase_chain: 오직 대화 맥락을 읽고 '질문 교정'만 전담 (답변 금지 지침 포함)

③ qa_chain: 오직 배달된 매뉴얼을 보고 '고객 응대'만 전담

④ 효과: 한 개의 LLM에게 "맥락도 고치고, 매뉴얼도 찾고, 친절하게 답해라"고 하면 과부하가 걸려 말을 안 듣지만, 이렇게 임무를 쪼개주면 gpt-4o-mini급의 가벼운 모델로도 상용 성능을 낼 수 있음

LangGraph 에이전트: 멀티 에이전트 적용 실습

- 고객 응대 Chatbot Agent 업그레이드(3)



실습: 싱글 에이전트 업그레이드 & 멀티 에이전트 RAG 시스템 구축

- 실습 개요

- 목표

1) 멀티 에이전트 협업 : 목표(고객 응대)를 해결하기 위해 '질문 재구성 담당'과 '문서 검색 담당' 그리고 '답변 생성 담당' 에이전트가 어떻게 역할을 분담하고 순차적으로 협업하는지 그 구조를 이해한다
2) 상태 기반의 데이터 흐름 설계: 한 에이전트의 작업 결과가 다음 에이전트로 자연스럽게 전달되고, 작업 흐름이 제어되는 방식을 파악한다
3) 조건부 라우팅(Conditional Routing) 구현: 조건부 분기를 사용하여 상태의 값에 따라 다음에 실행될 노드를 동적으로 결정하는 라우팅 로직을 구현한다
4) 비동기 실행 구현: async / await 문법을 활용하여 그래프의 노드와 전체 실행 흐름을 비동기적으로 처리한다.
5) 체크포인팅 구현: AsyncSqliteSaver를 체크포인터(Checkpointer)로 사용하여 대화의 중간 상태와 이력을 데이터베이스에 저장하고, 대화의 연속성을 유지하는 체크포인팅을 구현한다

실습: 싱글 에이전트 업그레이드 & 멀티 에이전트 RAG 시스템 구축

- 실습 개요
  - 과정
    1) API 키 로드
    2) RAG 시스템 기반 설정
    3) 멀티 에이전트 시스템의 그래프 상태 정의
    4) 비동기 노드 함수(전문가 에이전트) 정의
    5) 그래프 구성: 그래프 생성, 노드 추가, 시작점 설정, 엣지 추가, 컴파일
    6) 체크포인팅 및 맥락 유지 기능 구현
    7) 비동기 실행

실습: 싱글 에이전트 업그레이드 & 멀티 에이전트 RAG 시스템 구축



PydanticAI Agent를 이용한 LLM 출력 구조화의 혁신

- 예시: '자재 재고 분석 및 자동 발주 에이전트'

  사용자가 품목에 대한 재고 분석을 요청하면, 자재 데이터를 추출/확인하여 기준치 미달 시 자동으로 주문 수량까지 계산해 주는 B2B 공정 자동화 에이전트

사용자 요청: **"현재 창고 확인 결과 고강도 볼트M8의 수량이 부족하여 경고 알림(실재고는 현재 15개 남음)"**



PydanticAI Agent를 이용한 LLM 출력 구조화의 혁신

- **시나리오 1 (실패)**

: 연구/분석 Node LLM의 무질서한 답변 생성

1) 텍스트 파싱 중 에러가 나거나,

2) 수량 자리에 문자열 "수량 부족 (재고 15개)"이 들어와서 int(" 수량 부족 (재고 15개)") 연산을 하다가 ValueError로 시스템 에러 발생(error_message에 에러 기록)

```
llm_bad_output = (
    "네, 요청하신 자재 재고 분석 결과입니다!\n"
    "```json\n"
    "{\n"
    "  'item_name': '고강도 볼트M8',\n"
    "  'current_stock': '수량 부족 (재고 15개)'\n"
    "}\n"
    "```\n"
    "추가 조치가 필요하면 말씀하세요."
)
```

PydanticAI Agent를 이용한 LLM 출력 구조화의 혁신

- **시나리오 2 (성공)**

: 연구/분석 Node LLM: 반드시 지켜야 하는 사전에 약속된 형식의 답변 생성

1) 무조건 정수(int) 타입임이 보장된 데이터를 전달받고,

2) 에러 없이 100 - 15 = 85개 주문이라는 정확한 연산 결과를 도출하고 안전하게 프로세스를 마침

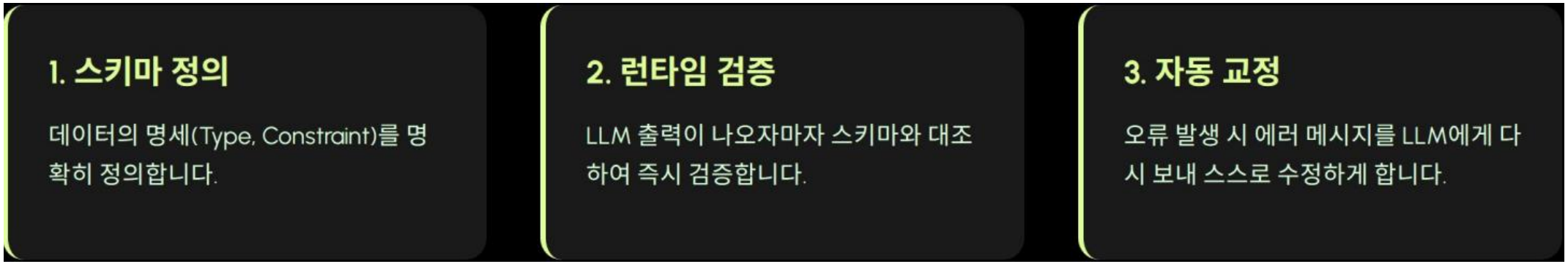
```
structured_data = {'item_name': '고강도 볼트M8', 'current_stock':15}
```

PydanticAI Agent를 이용한 LLM 출력 구조화의 혁신

- 구원투수: PydanticAI Agent
  1) 자연어(비정형)를 다루는 LLM에게 '반드시 지켜야 하는 출력 데이터 형식''을 강제
  2) 다음 시스템 노드가 안심하고 프로그래밍 연산을 수행할 수 있도록 데이터 신뢰성을 보장

PydanticAI Agent를 이용한 LLM 출력 구조화의 혁신

- PydanticAI Agent의 핵심 로직



실습: PydanticAI Agent를 이용한 LLM 출력 구조화의 혁신



25-2. 서울시 상권 분석 및 보고서 작성

멀티 에이전트 개발

개발 시나리오 1단계

- 데이터 파이프라인 구축

1) 목표: AI가 정확한 NL2SQL을 수행할 수 있도록 고품질의 최신 상권 데이터를 확보하고 구조화
2) 세부 과업: 서울 열린데이터광장 API를 활용한 데이터 수집 자동화
3) 결과물: seoul_commercial_sales_db.py

실습: 개발 시나리오 1단계 구현



개발 시나리오 2단계

- 핵심 로직 개발 (Core Logic)

1) 목표: PydanticAI Agent와 LangGraph를 결합한 4단계 지능형 분석 노드 구현

2) 세부 과업

① 질문 재구성 노드 (Rephrase): 이전 대화 내역을 분석하여 "그럼 거기와 흑리단길 상권을 비교할 경우 2024년도 여성 매출액은 어때?"를 "2024년도 성수동카페거리와 흑리단길 상권의 여성 매출액을 비교해 주세요"로 구체화

② SQL 생성 노드 (NL2SQL): 재구성된 질문과 DB 스키마를 결합하여 실행 가능한 SQLite 쿼리 자동 생성

③ SQL 실행 노드 (Execution): 생성된 쿼리를 asyncio.to_thread로 안전하게 실행하여 데이터 추출

④ 보고서 생성 노드 (Report): 분석 결과와 사용자 의도를 결합하여 비즈니스 통찰력이 담긴 마크다운 보고서 작성

3) 결과물: data_analysis_agent.py

실습: 개발 시나리오 2단계 구현



개발 시나리오 3단계

- 구조화된 바이브 코딩을 이용한 웹 APP 개발

1) 목표: 사용자가 AI 에이전트와 효율적으로 상호작용할 수 있는 웹 APP을 구현합니다

2) 세부 과업: gemini와 같은 생성형 AI를 이용하여 개발한 코드와 개발 시나리오를 제시하고 로컬에서 작동하는 웹 APP을 개발한다.

3) 결과물: app.py

실습: 개발 시나리오 3단계 구현



25-3. MCP

(Model Context Protocol)

기업용 AI 애플리케이션의 진화



단일 에이전트

- 특정 데이터를 참조하고, 정해진 도구를 사용하여 비교적 간단한 질문에 답변하거나 정형화된 업무를 수행
- 유능한 개인
- 정확성과 효율성에 명백한 한계

멀티 에이전트

- 복잡한 문제를 여러 개의 작은 단위로 나누어 여러 전문 에이전트를 정의하고 서로 협업
- 전문가 팀
- 모든 에이전트와 도구가 하나의 애플리케이션 내에서 실행되는 모놀리식 (Monolithic) 구조

- **확장성의 한계:** 특정 에이전트의 부하가 급증할 때, 해당 부분만 독립적으로 확장하기 어렵고 전체 시스템에 영향을 줌
- **관리의 어려움:** 에이전트의 종류가 많아지고 작업 흐름이 복잡해질수록, 전체 시스템의 상태를 파악하고 디버깅하기가 매우 까다로워짐

기업용 AI 애플리케이션의 진화: LangGraph를 넘어 MCP가 필요한 이유

- AI 기능을 독립적인 서비스로 분리하고, MCP라는 표준화된 방식으로 이들을 연결
  1) 비유: 한 사무실에서 일하는 것이 아니라, 각자 독립된 건물을 가진 전문 부서(서버)들이 표준화된 통신망(MCP)을 통해 협력하는 거대한 '기업 조직'을 만드는 것
  2) 독립적인 네트워크의 구축

- 장점
  1) 독립적인 확장: "보고서 서버만 늘리면 됩니다."
     ① 보고서 생성 요청이 폭주하면, 오직 '보고서 생성 서버'만 필요한 만큼 증설하면 됨
     ② 비용 효율적인 탄력적 확장 가능
  2) 기술의 자유: "각 부서는 최고의 도구로 일합니다."
     ① '시장 분석 서버'는 Python으로, '결제 처리 서버'는 Java로 만들 수 있음
     ② 각 기능에 가장 적합한 최고의 기술을 자유롭게 선택하고 조합 가능

서버-클라이언트 구조

- 특징: 역할에 따른 분리
- 예시: 은행 앱을 이용한 스마트 뱅킹

1) 클라이언트 (Client)

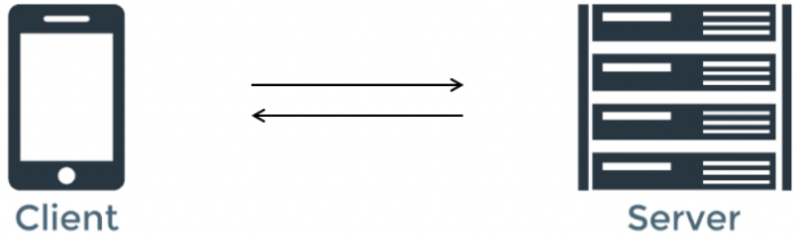
① 고객의 손안에 있는 '은행 앱

② 역할: 은행 서비스(잔액 조회, 이체 등)를 요청(Request)하는 프로그램

2) 서버 (Server):

① 은행의 '중앙 컴퓨터 시스템

② 역할: 클라이언트의 요청을 받아 실제 업무를 처리하고 결과를 제공(Response)하는 컴퓨터 시스템



서버-클라이언트 구조

3) 서버와 클라이언트를 분리하는 가장 큰 이유

① 역할 분담을 통한 효율성 극대화

② 독립적인 개발과 업데이트 (독립성): 은행은 고객용 앱(클라이언트) 디자인을 바꾸거나 새로운 기능을 추가할 때, 은행의 핵심 중앙 시스템(서버)을 멈추지 않아도 됩니다.

③ 안정적인 서비스 제공 (확장성): 월급날처럼 접속자가 한꺼번에 몰릴 때, 은행은 중앙 시스템(서버)의 처리 용량만 일시적으로 늘려서 모든 요청을 감당할 수 있습니다. 우리 각자의 스마트폰(클라이언트) 성능과는 관계없이 안정적인 서비스가 가능합니다.

④ 명확한 책임과 보안 (관리 용이성): 만약 이체 오류가 발생했다면, 그것이 내 스마트폰 앱의 문제인지 은행 중앙 시스템의 문제인지 책임 소재를 명확히 하여 신속하게 해결할 수 있습니다.

⑤ 요청하는 쪽(클라이언트)과 요청을 처리하는 쪽(서버)을 명확히 분리함으로써, 은행 시스템은 수천만 명이 사용하는 거대하고 복잡한 서비스를 안정적이고 효율적으로 운영할 수 있음

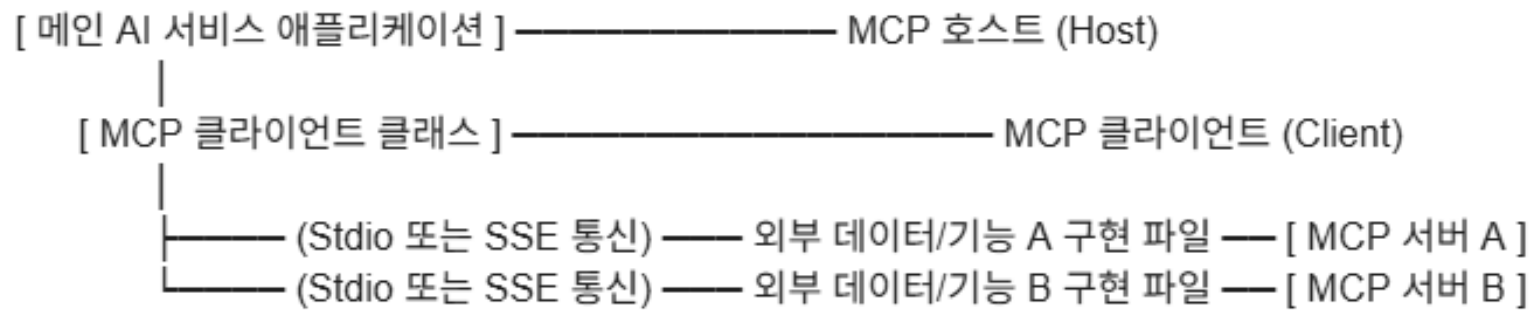
MCP(Model Context Protocol) 개요

- 정의: LLM과 외부 세계를 잇는 표준 통신 규약이자 시스템
  1) 거대 언어 모델(LLM)이 외부 세계와 지능적으로 상호작용할 수 있도록 컨텍스트(Context: 도구, 리소스, 프롬프트)를 표준화된 방식으로 주고받는 통신 규약이자 시스템
  2) LLM이 단순히 텍스트를 생성하는 것을 넘어, 외부 환경을 인지하고 행동할 수 있는 기반 마련

- 역할 및 기능: LLM의 능력 확장
  1) 유용성 및 정확성 향상: 실시간 정보, 특정 도메인 지식 등 외부의 최신 컨텍스트를 활용하여 더 정확하고 시기적절하며 관련성 높은 응답 생성
  2) 행동 역량 강화: 외부 시스템의 다양한 도구(API, 데이터베이스, 소프트웨어 기능 등)를 호출하고 제어하여 실제 세계에서 특정 작업 수행

MCP(Model Context Protocol) 아키텍처

- 3계층 시스템 구조
  - MCP는 역할 분리와 데이터 보안 격리를 위해 MCP 호스트(Host), MCP 클라이언트(Client), MCP 서버(Server)의 3계층 구조로 구동됨



MCP(Model Context Protocol) 아키텍처

- 3계층 핵심 역할 요약

1) MCP 호스트 (Host): AI 애플리케이션 실행 환경 (LLM 오케스트레이션 및 보안/권한 통제)

2) MCP 클라이언트 (Client): Host 내부에서 여러 MCP 서버와의 세션 연결 및 메시지 라우팅 전담

3) MCP 서버 (Server): 특정 데이터나 기능을 MCP 표준 규격으로 제공하는 독립형 프로그램

MCP(Model Context Protocol) 구성요소

- MCP 호스트 (Host)

1) 정의: 사용자가 직접 대화하고 작업하는 최상위 AI 서비스 및 애플리케이션 실행 환경

2) 주요 역할 및 기능:

① 클라이언트 생명주기 관리: 필요한 MCP 클라이언트 클래스를 생성하고 서버 연동 통제

② LLM 오케스트레이션: LLM과의 대화 흐름을 관리하고, 전달받은 컨텍스트를 추론 과정에 결합

③ 보안 및 사용자 승인(Consent) 통제: 외부 도구 호출이나 데이터 전송 시 사용자의 승인 절차를 거치며 접근 권한 제어

MCP(Model Context Protocol) 구성요소

- MCP 클라이언트 (Client)

1) 정의: Host 프로세스 내에 위치하며, 하나 이상의 MCP 서버와 통신 세션을 맺고 연결을 통합 관리하는 클래스

2) 주요 역할 및 기능:

① 다중 서버 세션 및 라우팅 관리: 여러 개의 MCP 서버(서버 A, 서버 B 등)와 동시에 연결 세션을 맺고 적절한 서버로 라우팅

② 양방향 프로토콜 처리: Host의 요청을 표준 JSON-RPC 메시지로 변환하여 서버로 전송하고, 서버의 응답을 수신

③ LLM의 한계 극복 지원: 실시간 정보 접근, 외부 데이터베이스 연동, 시스템 제어 한계를 우회하도록 지원

MCP(Model Context Protocol) 구성요소

- MCP 서버 (Server)

1) 정의: 외부 세계의 다양한 정보와 기능을 표준화된 '컨텍스트' 형태로 제공하는 독립 연결 통로

2) 주요 역할 및 기능:

① 컨텍스트 제공: 외부 데이터 및 기능을 '도구', '리소스', '프롬프트' 형태로 변환하여 제공

② 외부 시스템 연동: 실제 데이터베이스, API, 파일 시스템과 직접 통신하여 요청을 처리하고 결과 반환

③ 독립성 및 보안 격리 (Isolation): 각 서버는 자신에게 지정된 기능만 수행하며, 전체 대화 기록이나 타 서버의 정보를 알 수 없어 보안성 확보

④ 다양한 전송 방식 지원: 로컬 환경 통신(stdio) 및 원격 네트워크 통신(Streamable HTTP) 지원

MCP(Model Context Protocol) 주요 용어

- 컨텍스트

1) 정의: LLM이 주어진 작업을 수행하거나 질문에 답하는 데 필요한 모든 정보와 실행 가능한 기능으로 MCP 서버와 클라이언트 간의 통신에서 주고받는 핵심 '내용물'

2) 세 가지 주요 형태

① 도구(Tools): LLM이 특정 작업을 수행하기 위해 호출할 수 있는 함수나 API (예: 웹 검색, DB조회, 계산)

② 리소스(Resources): LLM이 참조할 수 있는 정적 또는 동적 데이터 (예: 문서, 이미지, 파일 내용)

③ 프롬프트 (Prompts): LLM에게 주어지는 구체적인 지시나 예시, 대화 흐름을 정의하는 템플릿

MCP(Model Context Protocol) 주요 용어

- 통신 규약: JSON-RPC 2.0

1) 정의: 서버와 클라이언트 간에 요청(Request)과 응답(Response)을 약속된 JSON 데이터 구조로 정의한 표준 규약

2) 예시

```
{
  "jsonrpc": "2.0",
  "method": "subtract",
  "params": [42, 23],
  "id": 1
}
```

도구 호출 요청 (Request)


```
{
  "jsonrpc": "2.0",
  "result": 19,
  "id": 1
}
```

성공 응답 (Response)

```
{
  "jsonrpc": "2.0",
  "error": {
    "code": -32601,
    "message": "Method not found"
  },
  "id": 1
}
```

에러 응답 (Error Response)

MCP(Model Context Protocol): 표준화된 연결의 중요성

- 상호 운용성 (Interoperability) - "AI 시대의 USB-C":

  1) 규격을 표준화함으로써 어떤 LLM 및 AI 애플리케이션이든, 어떤 MCP 서버든 서로 쉽게 연동 가능

  2) 다양한 기기의 데이터 전송과 충전을 단일 규격으로 처리하는 USB-C처럼 통신 방식을 하나로 통합

- 유연한 다중 서버 확장성:

  • 클라이언트가 여러 MCP 서버를 동시 연결하여 제어할 수 있으므로, 메인 앱의 수정 없이 새로운 외부 시스템을 언제든 독립적으로 추가·확장 가능

- 개발 용이성:

  • 애플리케이션 개발자와 데이터/도구 제공자가 독립적으로 개발할 수 있어 연동 복잡성이 대폭 감소

MCP 실습 환경 구축

- 필요한 라이브러리 설치

```
pip install fastmcp langchain-mcp-adapters
```

실습: FastMCP를 이용한 단일 서버 - 클라이언트 구현

- 애플리케이션 구조 개요(Architecture Overview)
  - 클라이언트 (client_architecture.py)
    1) 역할: "고객" 또는 "업무 지시자"
    2) 동작: 서버 프로세스를 직접 실행시키고, '표준 입출력(stdio)'을 통해 서버에게 작업을 지시하고 결과를 받는다.

  - 서버 (tool_server_architecture.py)
    1) 역할: "전문가"
    2) 동작: 클라이언트에 의해 실행되며, 오직 연결 통로로 들어오는 JSON 요청(주문서)에 따라서만 자신의 전문적인 작업(도구 함수: 인사말 작성)을 수행하고 결과를 전달한다.

```
payload = {"input_data": {"name": "박안정", "language": "한국어"}}
```



```
# 성공 시
{"result": {"greeting": "안녕하세요, 박안정님!..."}}
# 실패 시
{"error": "지원하지 않는 언어입니다..."}
```

실습: FastMCP를 이용한 단일 서버 - 클라이언트 구현

- 실습 개요: 단일 전문가 서버와 클라이언트 구현

- 목표

1) FastMCP를 이용한 서비스(도구) 구현: Python 함수를 FastMCP 서버의 "도구(Tool)"로 외부에 안전하게 노출하는 방법을 학습한다.
2) 안정적인 서버 로직 구축: try...except 구문을 활용하여, 예측하지 못한 오류가 발생해도 서버가 중단되지 않고 안정적으로 에러 메시지를 반환하는 안정적인 코드를 구현한다.
3) Pydantic 데이터 계약 및 애플리케이션 데이터 교환 규칙 이해
   ① Pydantic의 BaseModel을 활용하여 MCP 서버가 제공하는 도구(Tool)의 입력 데이터 형식(JSON 스키마)을 명확히 규정한다.
   ② 이를 통해 데이터 생산자(서버)와 데이터 소비자(MCP 호스트 및 클라이언트) 간의 '데이터 계약'을 수립하고, {"input_data": ...} 및 {"result": ...}와 같은 애플리케이션 레벨의 표준 JSON 구조로 데이터를 안전하게 주고받는 흐름을 명확히 이해한다.
4) 클라이언트-서버 비동기 통신 이해: MultiServerMCPClient가 asyncio를 통해 어떻게 서버 프로세스를 실행하고, 비동기 방식(await)으로 요청과 응답을 주고받는지 그 흐름을 파악한다.

실습: FastMCP를 이용한 단일 서버 - 클라이언트 구현

- 실습 개요
  - 과정
    1) '인사 전문가' 서버 구현 (tool_server_architecture.py)
       ① FastMCP 객체를 GreetingServer라는 이름으로 생성하여 MCP 서버를 정의
       ② Pydantic의 BaseModel을 상속받아, name과 language를 필드로 갖는 GreetingInput 자료형을 정의
       ③ @mcp.tool() 데코레이터를 사용하여 create_greeting 함수를 MCP 도구로 등록
       ④ 함수 내에 try...except 블록을 구현하여, 지원하지 않는 언어가 입력될 경우 ValueError를 발생시키고, 그 외 모든 예외 상황에서도 안정적으로 {"error": ...} 딕셔너리를 반환하도록 설계
       ⑤ mcp.run()을 호출하여 MCP 서버를 실행하고, 클라이언트와 표준 입출력(stdio)을 통해 통신하도록 설정

실습: FastMCP를 이용한 단일 서버 - 클라이언트 구현

- 실습 개요
  - 과정

2) '업무 지시자' 클라이언트 구현 (client_architecture.py)

① 클라이언트 생성: MultiServerMCPClient 클래스 함수 실행, tool_server_architecture.py 파일이 백그라운드에서 실행되도록 설정

② await client.get_tools() 함수 호출: 서버가 제공할 수 있는 도구의 명세서(이름, 설명, 필요한 입력값 정보) 요청

③ MCP 요청 프로토콜에 따라 서버 도구 함수의 입력 스키마에 맞는 Python 딕셔너리 생성

④ json.loads() 함수를 사용하여 서버가 보낸 JSON 문자열을 다시 Python 딕셔너리로 변환

⑤ MCP 응답 프로토콜에 따라 응답 딕셔너리에 result 키가 있는지, error 키가 있는지를 확인하여 성공과 실패 케이스를 모두 처리하고 결과를 출력

실습: FastMCP를 이용한 단일 서버 - 클라이언트 구현



실습: FastMCP 멀티 서버 구축과 LangGraph 에이전트 통합 구현

- 실습 개요
  - 목표
    1) 역할 기반 마이크로서비스 아키텍처 이해: '시장 조사', '보고서 작성' 등 명확한 역할을 가진 전문가 기능들을 독립적인 FastMCP 서버로 분리하여 마이크로서비스(Microservices) 와 유사한 아키텍처를 구축하는 방법을 학습한다.
    2) LangGraph를 이용한 워크플로우 오케스트레이션: 중앙의 클라이언트가 LangGraph의 상태 기반 라우팅(Stateful Routing)을 사용하여, 여러 전문가 서버들을 정해진 순서에 따라 조율하고 전체 비즈니스 프로세스를 관리하는 방법을 이해한다.
    3) 상태 클래스(자료형)를 통한 데이터 전달: Pydantic BaseModel로 정의된 상태(OrchestratorState) 클래스가 어떻게 노드(에이전트) 간에 정보를 전달하는 '공동 작업 화이트보드' 역할을 하는지 파악한다.
    4) 견고한 비동기 통신 구현: MultiServerMCPClient를 통해 여러 개의 서버 프로세스를 동시에 관리하고, 각 서버의 도구를 비동기적으로 호출하여 결과를 받아오는 안정적인 통신 방식을 구현한다.
    5) 지속 가능한 대화 관리: AsyncSqliteSaver를 이용하여 에이전트의 작업 내역과 대화 기록을 데이터베이스에 저장하고, 이전 대화의 맥락을 이어갈 수 있는 지속성(Persistence) 있는 시스템을 구현한다.

실습: FastMCP 멀티 서버 구축과 LangGraph 에이전트 통합 구현

- 실습 개요
  - 과정

1) '시장 조사 전문가' 서버 구축 (market_research_server.py)
   ① FastMCP 서버를 MarketResearchExpert라는 이름으로 정의하고, TavilySearch 도구 설정
   ② 조사 주제(topic)를 입력 받는 ResearchInput 자료형 정의
   ③ onduct_market_research 함수를 MCP 도구로 노출
2) '보고서 작성 전문가' 서버 구축 (report_writing_server.py)
   ① FastMCP 서버를 ReportWritingExpert라는 이름으로 정의하고, 보고서 생성용 LLM 설정
   ② 사용자 원본 질문(user_query)과 시장 조사 요약본(research_summary)을 입력 받는 ReportInput 자료형 정의
   ③ write_final_report 함수를 MCP 도구로 노출

실습: FastMCP 멀티 서버 구축과 LangGraph 에이전트 통합 구현

- 실습 개요
  - 과정

3) '중앙 지휘 Orchestrator' 클라이언트 구축 (multiserver_client.py)

① MultiServerMCPClient를 설정하여 두 개의 전문가 서버를 실행하고 관리

② Pydantic BaseModel을 사용하여, 대화 기록(messages), 조사 결과(research_summary), 원본 질문(user_query), 다음 목적지(next_node)를 포함하는 OrchestratorState 정의

③ supervisor, market_research, report_writing 역할을 하는 3개의 비동기 노드 함수를 정의, 각 노드는 자신의 역할을 수행한 후, 상태의 next_node 필드를 업데이트하여 다음 목적지를 명시적으로 기록

④ router 함수를 정의하여, 상태의 next_node 값을 읽고 다음 노드로 흐름을 분기

⑤ StateGraph를 생성하고, 노드와 엣지를 연결하여 전체 작업 워크플로우 구성

⑥ AsyncSqliteSaver를 checkpointer로 설정하여 그래프를 컴파일함으로써, 모든 상태 변화를 SQLite 데이터베이스에 자동 저장

실습: FastMCP 멀티 서버 구축과 LangGraph 에이전트 통합 구현




MISSION:
COMPLETE